

UNIVERSIDAD PERUANA UNIÓN



Una Institución Adventista

INGENIERÍA DE SISTEMAS

Ciclo VII GRUPO-1

Ingeniería de software II

Profesor responsable: Ing. Ruben Roque Sucari

INFORME DE TRABAJO FINAL

Nombre del startup: CoreSystems Solutions

Nombre del producto: AstraLog – Solución Logística Centralizada

Grupo: 5

Integrantes:

Elvis Jesus Apaza Yucra - 202312728

Anthony Kelman Mamani Vargas – 202312049

Yunior Benito Quispe Quispe - 202312058

Jeimy Paul Ramos Coaquira – 202310096

Juliaca, mayo de 2026

Índice

1 INTRODUCCIÓN	4
2 DESCRIPCIÓN DEL NEGOCIO	5
3 STAKEHOLDERS DEL SISTEMA	6
4 ENFOQUE DE MODELADO ARQUITECTÓNICO	6
4.1 Niveles de modelado utilizados	6
4.2 Vista de contexto (nivel 1)	7
4.3 Elementos	7
4.4 Consideraciones	7
4.5 Vista de contenedores (nivel 2)	8
4.5.1 Contenedores	8
4.5.2 Responsabilidades	8
4.5.3 Consideraciones	9
4.6 Vista de componentes (nivel 3)	9
4.6.1 Módulo de Ventas	10
4.6.2 Módulo de Inventario	10
4.6.3 Módulo de Autenticación	10
4.6.4 Módulo de Usuarios	10
4.6.5 Módulo de Auditoría	10
4.6.6 Consideraciones	10
5 LÍMITES Y RESPONSABILIDADES	14
5.1 Consideraciones	14
6 ATRIBUTOS DE CALIDAD	15
7 APLICACIÓN DE PRINCIPIOS SOLID	16
8 ESTILO ARQUITECTÓNICO	17
8.1 Fundamentación	17
8.2 Estructura	17
8.3 Estructura	17
8.4 Trade-offs	18
9 COHERENCIA DEL DISEÑO	19
10 REPO DE GITHUB	19
11 ELIMINADO LÓGICO	19
12 K6 - IMÁGENES DEL PROYECTO	22
13 TRABAJO COLABORATIVO GITHUB APROBADO CON 2 REVISORES	27
14 CMMI+Scrum utilizando JIRA (las 3 áreas TS, VER, IP)	30
15 REFERENCIAS	51
16 ANEXOS	52

16.1 ANEXO A: CÓDIGO PLANTUML – DIAGRAMA DE CONTEXTO (C4 NIVEL 1)	52
16.2 ANEXO B: CÓDIGO PLANTUML – MOLITO - VENTAS (C4 NIVEL 2)	52
16.3 ANEXO C: CÓDIGO PLANTUML – MONOLITO DE AUTENTICACIÓN (C4 NIVEL 3)	53
16.4 ANEXO D: CÓDIGO PLANTUML – MONOLITO DE USUARIOS (C4 NIVEL 3)	54
16.5 ANEXO E: CÓDIGO PLANTUML – AUDITORÍA (C4 NIVEL 3)	54
16.6 ANEXO F: CÓDIGO PLANTUML – Representación conceptual de la arquitectura de monolito (C4 NIVEL 3)	55

1 INTRODUCCIÓN

El presente documento técnico tiene como objetivo principal definir, estructurar y documentar formalmente la arquitectura de software del sistema de gestión de transporte, asignación equitativa y control logístico denominado “**AstraLog**”, desarrollado para la empresa de transportes de agregados y materiales de construcción **ASTRAMACO III**. El diseño arquitectónico expuesto se fundamenta en la aplicación rigurosa de principios de diseño estructural y la descomposición modular orientada al dominio, modelando la totalidad del ecosistema a través de vistas arquitectónicas estandarizadas mediante el Modelo C4 (Contexto, Contenedores y Componentes).

En este documento se establecen con precisión las vistas arquitectónicas que componen la solución, los límites del sistema, las responsabilidades detalladas de cada uno de sus componentes de software (Backend en Java Spring Boot, interfaces frontend web y aplicaciones móviles) y la justificación técnica del estilo arquitectónico seleccionado basado en monolito con persistencia aislada. Asimismo, se incorporan consideraciones de diseño crítico como el patrón de auditoría avanzada con serialización JSON y la estrategia de seguridad para la mitigación de vulnerabilidades en la gestión de sesiones JWT bajo la norma ISO/IEC 12207.

A través de esta especificación, se busca garantizar que el sistema "AstraLog" responda con solidez a los atributos de calidad de mantenibilidad, tolerancia a fallos, escalabilidad horizontal y alta cohesión exigidos en entornos de producción de Ingeniería de Software, sirviendo como el plano de ingeniería definitivo para el despliegue, mantenimiento y evolución de la plataforma informática.

2 DESCRIPCIÓN DEL NEGOCIO

La empresa **ASTRAMACO III** se dedica al transporte pesado y distribución de materiales de construcción (ladrillos, arena y agregados). Actualmente, la gestión de su flota de transportistas se realiza de forma manual y empírica, lo que genera problemas operativos críticos:

- **Tarifas ambiguas:** Las cotizaciones de fletes se calculan al cálculo, omitiendo variables clave como la distancia exacta de la ruta, el volumen del material y la dificultad de descarga por pisos en el destino.
- **Asignación injusta de viajes:** Los choferes deben esperar clientes físicamente en zonas de alto tránsito. Al no existir un orden transparente, se genera un desequilibrio de ingresos entre los transportistas.
- **Falta de control en ruta:** Las entregas se validan con guías físicas de papel. No hay un registro digital en tiempo real ni evidencias como fotos o coordenadas de geolocalización al descargar el material.

Para resolver estas deficiencias, la startup **CoreSystems Solutions** desarrolla el sistema **AstraLog**, una solución integral orientada a cumplir los siguientes objetivos funcionales:

1. **Cotización Automatizada:** Calcular tarifas exactas según distancia, volumen de carga y nivel de descarga.
2. **Asignación Equitativa:** Distribuir los viajes automáticamente priorizando la disponibilidad y el balance de ventas semanales de los choferes-
3. **Validación Digital (PoD):** Emitir hojas de ruta en la app móvil y exigir token del cliente para validación de entregas.
4. **Restricciones de Carga:** Controlar mediante software la capacidad máxima por vehículo y prohibir la mezcla de materiales incompatibles (ej. arena y ladrillos sueltos).
5. **Auditoría Integral:** Monitorear estados de pedidos, comisiones y registrar de forma indeleble los cambios mutables del sistema en un módulo central de logs.

3 STAKEHOLDERS DEL SISTEMA

Para garantizar que la arquitectura de software responda a las necesidades reales del negocio, se definen los actores clave (Stakeholders) que interactúan de manera directa o indirecta con la plataforma **AstraLog**.

Stakeholder	Descripción	Interacción con el Sistema	Canal de Acceso
Administrador Logístico	Personal encargado de la gestión operativa y de control en las oficinas de ASTRAMACO III.	Registra pedidos, gestiona la flota de vehículos, configura restricciones de carga y supervisa las hojas de ruta.	Aplicación Web (PC)
Transportista (Chofer)	Operador encargado de conducir los vehículos pesados y trasladar los materiales.	Recibe alertas de viajes asignados, visualiza su hoja de ruta y pide token del cliente para validación del pedido.	Aplicación Móvil (Smartphone)
Dueño / Gerencia	Propietario de la empresa, interesado en la rentabilidad y la transparencia operativa.	Supervisa el balance semanal de ventas, audita las comisiones generadas y evalúa los KPIs de rendimiento de la flota.	Aplicación Web (PC) y Móvil
Cliente	Persona natural o jurídica que solicita el servicio de flete de agregados o materiales.	Interacción indirecta. Recibe la cotización digital automatizada calculada por el sistema según los parámetros de su destino.	Ninguno (Interacción Indirecta)

4 ENFOQUE DE MODELADO ARQUITECTÓNICO

Para representar de forma clara la arquitectura del sistema AstraLog, se utiliza el modelo de vistas basado en **C4 (Context, Containers, Components)**. Este enfoque permite describir la solución tecnológica desde el nivel más general hasta el detalle del código interno del software.

4.1 Niveles de modelado utilizados

- Nivel 1 – Contexto:** Muestra cómo interactúa el sistema AstraLog con los usuarios y actores externos del negocio.

- b) **Nivel 2 – Contenedores:** Detalla la estructura general del sistema, especificando las aplicaciones frontend, móviles, el backend de microservicios y sus bases de datos aisladas.

4.2 Vista de contexto (nivel 1)

El sistema AstraLog centraliza las solicitudes de pedidos logísticos de agregados y automatiza la distribución del trabajo para los transportistas de ASTRAMACO III. Actúa como el núcleo operativo del negocio, comunicando a la administración con el personal en ruta.

4.3 Elementos

- a) **Sistema:** AstraLog (Core de gestión de fletes y transporte).
- b) **Actores Directos:** Administrador Logístico, Transportista (Chofer) y Dueño / Gerencia.
- c) **Actores Indirectos:** Clientes (receptores finales de las cotizaciones y mercancía).

4.4 Consideraciones

El sistema unifica los flujos informativos. Los usuarios acceden a través de canales diferenciados según su rol operativo: los administradores operan desde computadoras de escritorio, mientras que los transportistas en ruta utilizan dispositivos móviles inteligentes.

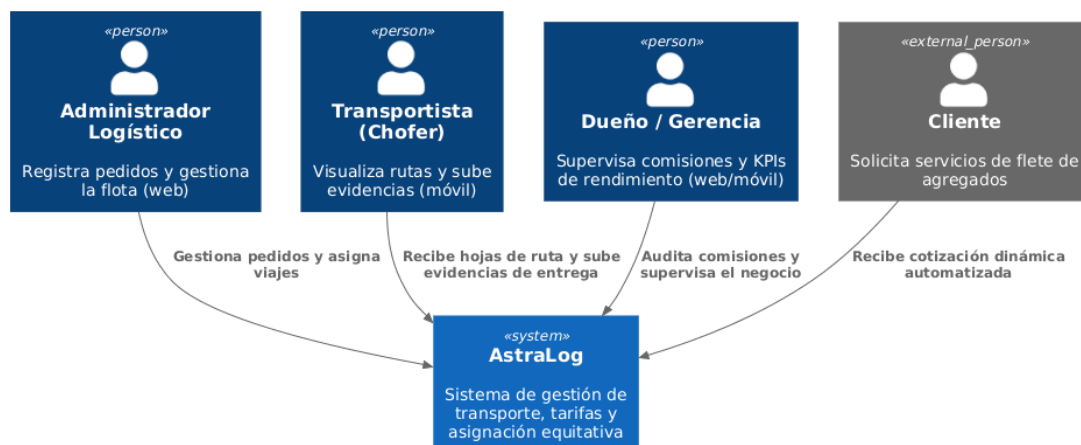


Imagen 1: Diagrama de contexto del AstraLog

Fuente: Elaboración propia (PlantUML)

Ver código en Anexo A

4.5 Vista de contenedores (nivel 2)

El sistema AstraLog está estructurado bajo un estilo arquitectónico de monolito para asegurar independencia operativa y aislamiento de fallos.

4.5.1 Contenedores

- a) **Aplicación Web (Frontend):** Interfaz gráfica para computadoras de escritorio utilizada por los administradores y la gerencia.
- b) **Aplicación Móvil (App):** Interfaz nativa o híbrida utilizada por los transportistas para recibir las hojas de ruta y subir evidencias.
- c) **API Gateway:** Punto único de entrada al backend que centraliza, autentica y rutea las solicitudes de los clientes.
- d) **Microservicios Backend (Spring Boot):** Bloques independientes de lógica de negocio (Autenticación, Usuarios, Pedidos, Flota/Inventario y Auditoría).
- e) **Bases de Datos Independientes:** Persistencia aislada por cada microservicio para evitar el acoplamiento de datos.

4.5.2 Responsabilidades

- i. **Aplicación Web:** Registro de pedidos, control administrativo de vehículos y monitoreo del panel de control empresarial.
- ii. **Aplicación Móvil:** Visualización de viajes asignados, actualización del estado de entrega en ruta y captura de evidencias (fotos y geolocalización).
- iii. **API Gateway:** Validación de tokens de seguridad y redirección del tráfico hacia los microservicios correspondientes.
- iv. **Monolito:** Procesamiento independiente de las reglas de negocio, tales como el algoritmo de asignación justa o el motor de cotización dinámica.
- v. **Bases de Datos:** Almacenamiento seguro y exclusivo de los datos correspondientes a cada dominio del negocio.

4.5.3 Consideraciones

En el sistema AstraLog, la arquitectura monolítica asegura la estabilidad y la integridad operativa bajo las siguientes premisas:

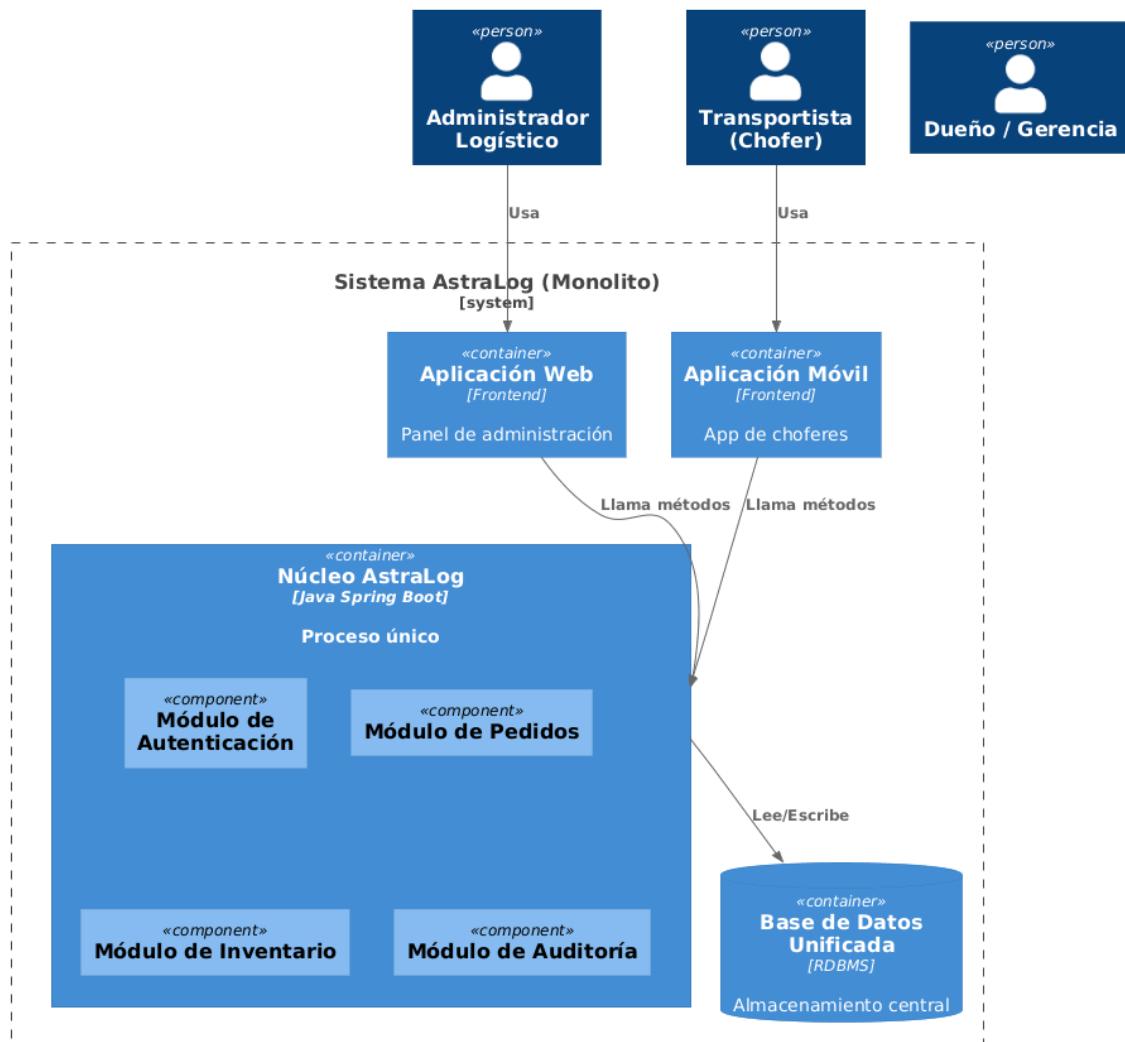


Imagen 2: Notación C4, diagrama de contenedores del AstraLog

Fuente: Elaboración propia (PlantUML)

Ver código en Anexo B

4.6 Vista de componentes (nivel 3)

La vista de componentes descompone el núcleo del backend (*AstraLog Monolith*) para analizar su estructura interna. A pesar de residir en un único proceso, los módulos siguen un patrón de diseño en capas (*Controller-Service-Repository*) dentro de sus respectivos paquetes lógicos para garantizar el bajo acoplamiento, la alta cohesión y la mantenibilidad a largo plazo.

4.6.1 Módulo de Ventas

- a) Registro y cotización dinámica de pedidos de fletes.
- b) Cálculo automatizado de tarifas según distancia, volumen y nivel de descarga.
- c) Algoritmo de asignación equitativa de viajes para los transportistas.

4.6.2 Módulo de Inventario

- a) Control de stock, tipos de agregados y materiales de construcción en almacén.
- b) Validación de capacidad máxima de carga por configuración vehicular.
- c) Restricción automatizada de mezcla de materiales incompatibles en tolva.

4.6.3 Módulo de Autenticación

- a) Validación segura de credenciales de usuarios y choferes.
- b) Generación y firma de tokens de seguridad (JWT).
- c) Validación y control de expiración de sesiones activas.

4.6.4 Módulo de Usuarios

- a) Gestión de perfiles de administradores, despachadores y transportistas.
- b) Control de acceso basado en roles (RBAC).
- c) Asignación y revocación de permisos específicos en el sistema.

4.6.5 Módulo de Auditoría

- a) Registro indeleble de logs y trazabilidad de operaciones mutables.
- b) Serialización en formato JSON de estados anteriores y nuevos ante cambios.
- c) Captura automatizada de metadatos de seguridad (direcciones IP y agentes de usuario).

4.6.6 Consideraciones

- a) Cada componente interno (Controller, Service, Repository) encapsula su propia lógica.
- b) Los componentes se comunican verticalmente sin saltarse capas de software.
- c) La interacción hacia almacenamiento de datos externos se realiza únicamente mediante su propio repositorio aislado.

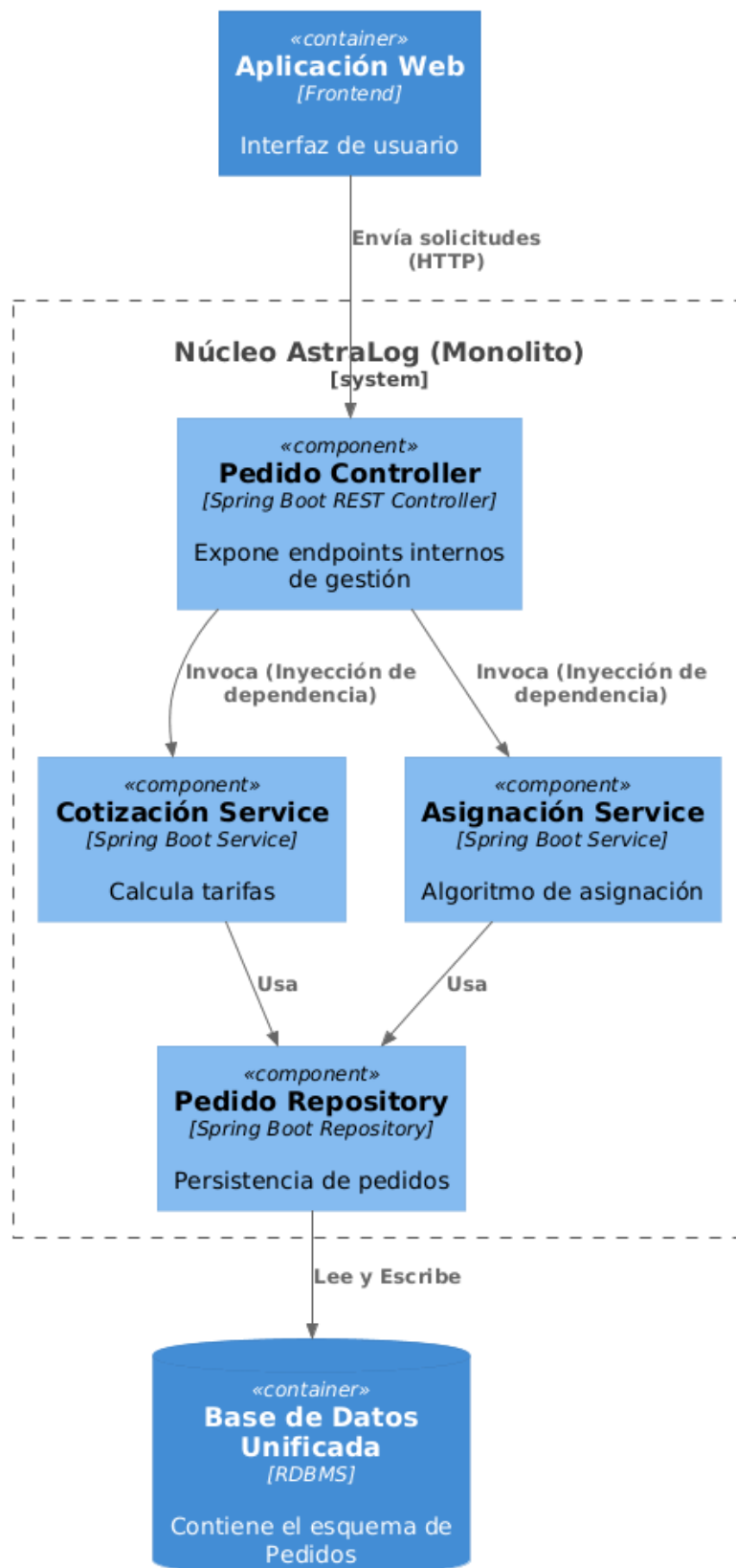


Imagen 3: Diagrama de componentes del Monolito de Pedidos (C4 Nivel 3)
Fuente: Elaboración propia (PlantUML)
Ver código en Anexo C

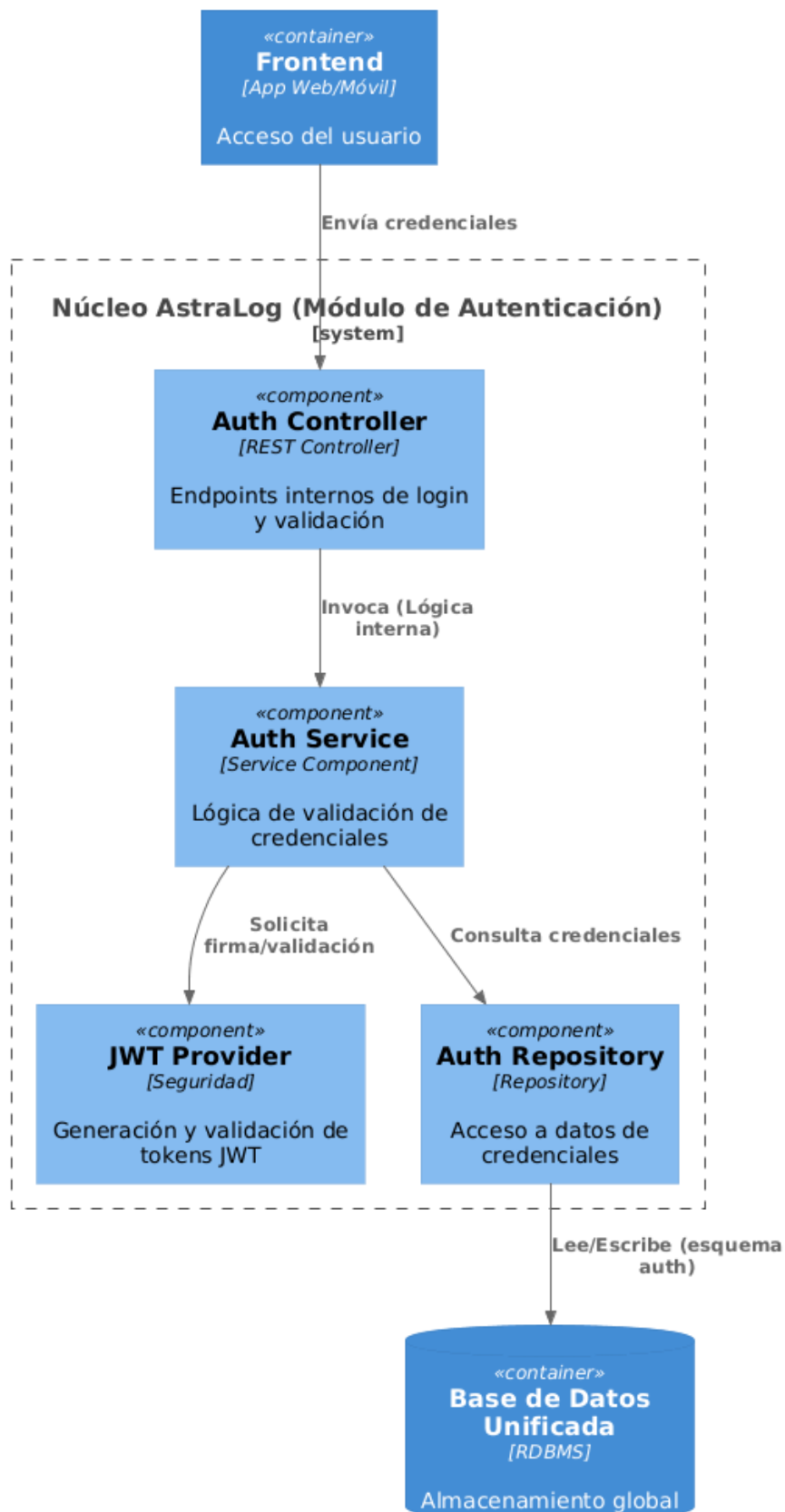


Imagen 4: Diagrama de componentes del Monolito de Autenticación (C4 Nivel 3)
Fuente: Elaboración propia (PlantUML)
Ver código en Anexo D

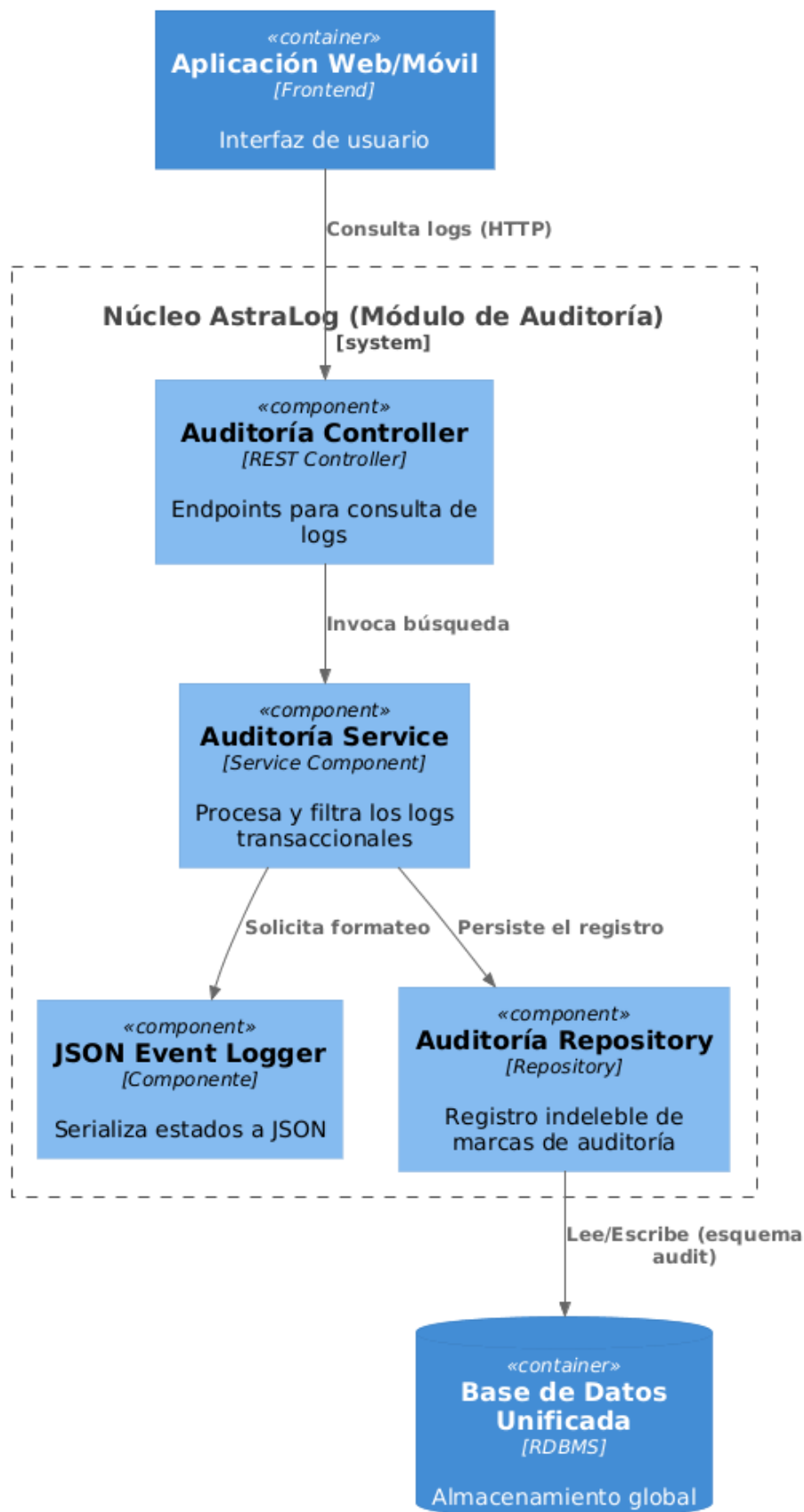


Imagen 5: Diagrama de componentes del Monolito de AUDITORÍA (C4 Nivel 3)
Fuente: Elaboración propia (PlantUML)
Ver código en Anexo E

5 LÍMITES Y RESPONSABILIDADES

La siguiente matriz detalla el alcance operativo y las fronteras técnicas del sistema **AstraLog**, permitiendo entender con claridad qué procesos cubre el software y cuáles quedan fuera de su alcance directo en **ASTRAMACO III**:

Responsabilidades del Sistema (Lo que SÍ hace)	Límites del Sistema (Lo que NO hace)
Cotización Dinámica de Fletes: Calcula de forma automática las tarifas en base a la distancia exacta, el volumen del material y la dificultad de la descarga.	Procesamiento de Pagos en Línea: El sistema no incluye pasarelas de pago ni transferencias; los flujos de dinero se gestionan externamente por administración.
Asignación Equitativa de Viajes: Controla automáticamente el orden de turnos de los choferes según su disponibilidad e histórico de ventas semanales.	Rastreo por Hardware GPS Integrado: El software no rastrea el camión físicamente. La geolocalización depende totalmente del smartphone del transportista.
Validación Digital de Entregas (PoD): Exige de forma obligatoria token del cliente y coordenadas geográficas en destino para poder dar por cerrada una ruta.	Soporte Fuera de Red (Offline Completo): Aunque permite guardar datos locales de forma temporal, requiere internet móvil o Wi-Fi para validar sesiones JWT y sincronizar las rutas con el servidor.
Control de Restricciones de Carga: Bloquea por software pedidos que superen el peso máximo permitido del vehículo o que mezclen materiales incompatibles en la tolva.	Gestión de Mantenimiento Mecánico: No realiza seguimiento de talleres mecánicos, compras de repuestos ni alertas de fallas internas de los motores de la flota.
Auditoría Transaccional Avanzada: Registra de manera permanente y transparente cada cambio de estado de los pedidos utilizando serialización de datos en formato JSON.	Despacho Autónomo Fuera de Criterios: El algoritmo no puede saltarse las reglas de negocio establecidas ni asignar viajes a conductores suspendidos o sin documentación al día.

5.1 Consideraciones

- Manejo de Contingencias en Ruta:** Ante cualquier falla mecánica o retraso por fuerza mayor en las carreteras, el transportista deberá reportar el incidente por canales externos (llamada telefónica o radio). El administrador logístico será el único facultado en la aplicación web para liberar manualmente el viaje y permitir que el algoritmo asigne el pedido a otro camión disponible.

- b) **Políticas de Expiración de Sesiones:** Como medida de protección de datos, los tokens de seguridad JWT configurados en el API Gateway cuentan con un tiempo de expiración estricto. Si la aplicación móvil del transportista pasa más de 24 horas sin conexión o inactiva, se forzará el cierre de sesión, exigiendo autenticación obligatoria antes de permitir nuevas cargas de evidencias.
- c) **Sincronización de Datos Post-Conectividad:** En zonas geográficas con nula cobertura de red, la aplicación móvil almacenará localmente en el dispositivo la captura fotográfica y las coordenadas capturadas por el GPS. Estos datos se enviarán automáticamente en segundo plano una vez que el dispositivo recupere una señal de red estable.
- d) **Responsabilidad del Soporte Técnico:** La startup CoreSystems Solutions asume la responsabilidad del correcto funcionamiento de la infraestructura lógica de los microservicios y la disponibilidad del backend. No obstante, el mantenimiento físico, las actualizaciones de sistemas operativos de los smartphones de los choferes y los planes de datos móviles corren por cuenta exclusiva de ASTRAMACO III.

6 ATRIBUTOS DE CALIDAD

Los atributos de calidad definen las propiedades y características estructurales que garantizan la solidez, escalabilidad y correcto funcionamiento de la arquitectura del sistema **AstraLog** para **ASTRAMACO III**:

- **Cohesión:** Monolito está diseñado para cumplir una única función específica dentro del negocio (por ejemplo, gestionar exclusivamente los pedidos o enfocarse únicamente en la auditoría), asegurando que los componentes internos están fuertemente relacionados con su tarea.
- **Bajo acoplamiento:** Monolito interactúa estrictamente mediante APIs REST estandarizadas, evitando dependencias directas entre ellos. La separación del microservicio de autenticación permite reducir el acoplamiento y mejorar la seguridad general del sistema, garantizando que un fallo en un módulo no detenga a los demás.
- **Modularidad:** El sistema está dividido en módulos independientes y autónomos. Esto facilita que se puedan realizar actualizaciones, mejoras o correcciones en un microservicio específico sin necesidad de modificar o redespigar todo el sistema de transportes.
- **Abstracción:** Se exponen únicamente las interfaces y datos que son estrictamente necesarios para la operación del usuario o la comunicación entre servicios. La complejidad interna de los algoritmos de tarifa o asignación queda totalmente oculta, ofreciendo conexiones más limpias y seguras.

7 APLICACIÓN DE PRINCIPIOS SOLID

Para garantizar que el código fuente del sistema **AstraLog** sea fácil de mantener, escalar y testear, se aplican los principios de diseño **SOLID** en la estructura de monolito desarrollados con Java Spring Boot:

- **S - Principio de Responsabilidad Única (Single Responsibility Principle):** Cada clase en el backend tiene una sola razón para cambiar. Por ejemplo, la clase **PedidoController** se encarga únicamente de recibir las solicitudes HTTP de transporte, mientras que la lógica del cálculo del flete se delega por completo a la clase **CotizacionService**.
- **O - Principio de Abierto/Cerrado (Open/Closed Principle):** Las entidades y componentes de software están abiertos a la extensión, pero cerrados a la modificación. Si en el futuro ASTRAMACO III decide agregar un nuevo tipo de tarifa (por ejemplo, tarifa nocturna), se puede extender la lógica creando una nueva estrategia de cálculo sin necesidad de alterar el código base que ya funciona en producción.
- **L - Principio de Sustitución de Liskov (Liskov Substitution Principle):** Las clases derivadas deben poder sustituir a sus clases base sin alterar el comportamiento del programa. En el sistema, cualquier implementación específica de un repositorio de datos puede reemplazar a la interfaz genérica de persistencia sin romper el flujo de las transacciones ni los accesos a la base de datos.
- **I - Principio de Segregación de Interfaces (Interface Segregation Principle):** Se prefieren interfaces pequeñas y específicas en lugar de una sola interfaz general. En AstraLog, un transportista utiliza una interfaz o servicio con métodos exclusivos para su rol (ver ruta y subir fotos), evitando que esté obligado a implementar métodos administrativos que no le corresponden, como la liquidación de comisiones o reportes gerenciales.
- **D - Principio de Inversión de Dependencias (Dependency Inversion Principle):** Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones. Mediante la inyección de dependencias de Spring Boot, los servicios del negocio (como **AsignacionService**) dependen de interfaces abstractas de repositorios y no de clases concretas de bases de datos, facilitando el cambio de tecnologías de almacenamiento de datos sin afectar la lógica del negocio.

8 ESTILO ARQUITECTÓNICO

Arquitectura de Monolito

8.1 Fundamentación

- a) **Separación de dominios:** El sistema se divide en contextos delimitados bien definidos (Autenticación, Usuarios, Pedidos, Flota y Auditoría), garantizando que cada módulo resuelva un problema único del negocio de fletes sin interferir en los demás.
- b) **Escalabilidad:** Permite dimensionar y asignar más recursos de forma independiente a los módulos que sufran mayor carga (como el motor de cotizaciones en horas punta de despacho) sin necesidad de replicar todo el sistema de software.

8.2 Estructura

Separación de responsabilidades críticas como autenticación, permitiendo mayor seguridad y escalabilidad del sistema, asegurando que la verificación de accesos no congestione la lógica principal de asignación de viajes.

8.3 Estructura

- a) **Núcleo AstraLog:** Aplicación central desarrollada en Java Spring Boot que actúa como un contenedor único de ejecución. Todas las reglas de negocio, anteriormente dispersas, coexisten en el mismo espacio de memoria, permitiendo una comunicación interna rápida mediante inyección de dependencias.
- b) **Punto de Entrada:** Se elimina el componente *API Gateway* externo; el sistema utiliza controladores web integrados que gestionan directamente las peticiones HTTP de los clientes hacia el módulo lógico correspondiente.
- c) **Persistencia Unificada:** Se utiliza una base de datos centralizada con esquemas lógicos aislados por módulo. Esto garantiza la integridad transaccional global (**ACID**) y simplifica la gestión de copias de seguridad y la consistencia de los datos, eliminando la complejidad de las transacciones distribuidas.

8.4 Trade-offs

- Mayor complejidad inicial:** Requiere un esfuerzo superior de diseño y configuración de infraestructura en comparación con un sistema monolítico tradicional, afectando las etapas iniciales del despliegue.
- Gestión de comunicación:** Al ser un sistema distribuido, introduce el reto de administrar la latencia de las peticiones de red y asegurar la sincronización de la información entre los microservicios sin romper la consistencia de los datos.

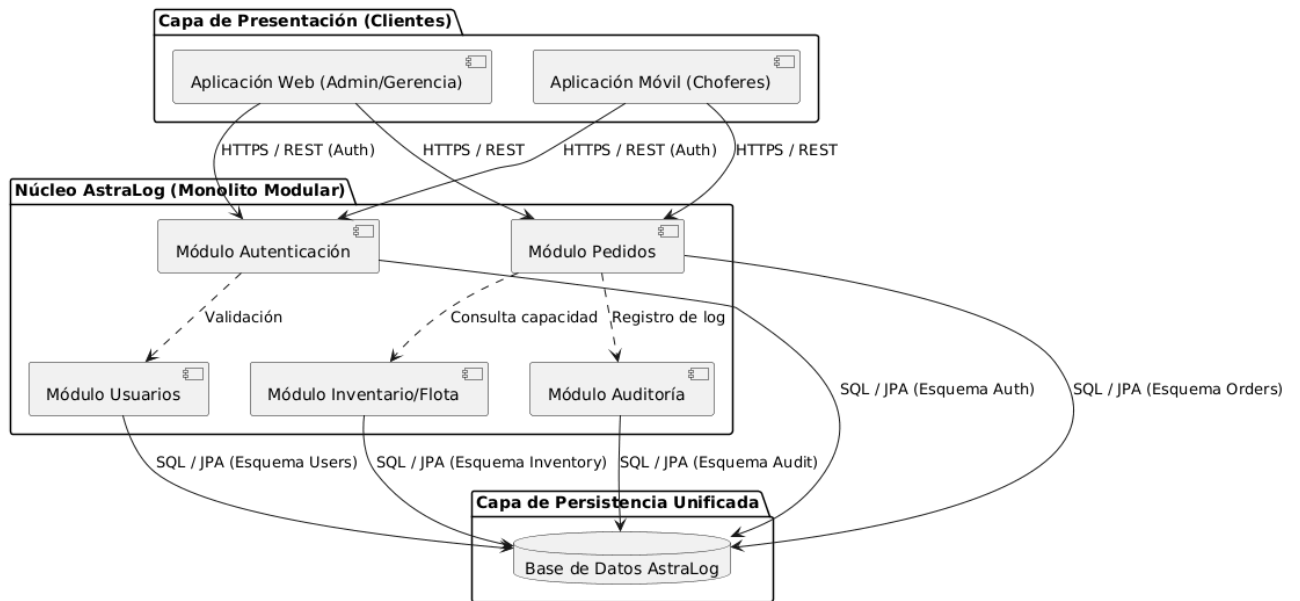


Imagen 8: Representación conceptual de la arquitectura de monolito

Fuente: Elaboración propia (PlantUML)

Ver código en Anexo F

9 COHERENCIA DEL DISEÑO

- a) Pedidos de fletes
- b) Cotización dinámica
- c) Asignación de turnos
- d) Control de stock de agregados
- e) Capacidad de carga vehicular
- f) Registro de logs transaccionales
- g) Serialización de estados en JSON
- h) Autenticación de usuarios y choferes
- i) Generación de tokens JWT
- j) Gestión de perfiles y usuarios
- k) Control de acceso basado en roles

10 REPO DE GITHUB

<https://github.com/elvisjsAtlas1/Backend-AstramacoIII.git> backend

<https://github.com/elvisjsAtlas1/AppAstraLog-AstramacoIII.git> app AstraLog

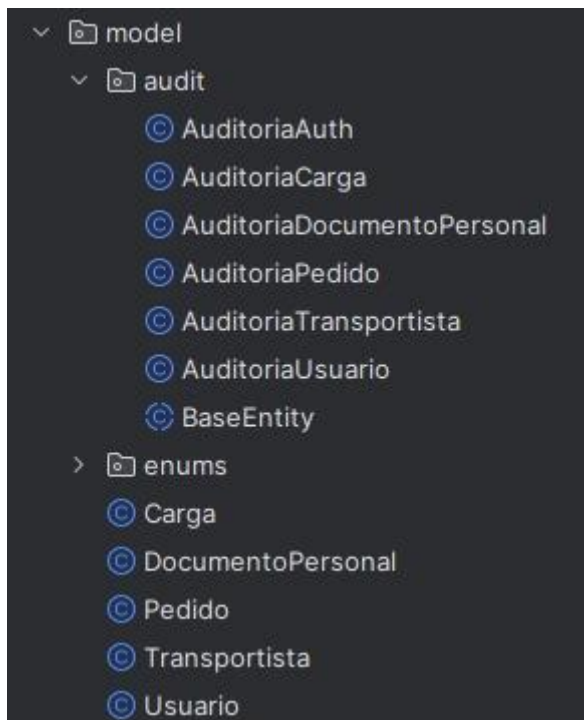
<https://github.com/elvisjsAtlas1/Frontend-AstramacoIII.git> Frontend

<http://localhost:8080/swagger-ui.html>

11 ELIMINADO LÓGICO



db creación de las auditorias por cada modulo



```
7  @Entity 10 usages elvisjsAtlas1
8  @Table(name = "auditoria_pedidos")
9  @Data
10 public class AuditoriaPedido {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     private Long pedidoId;
16     private String accion; // CREATE, UPDATE, DELETE
17     private String username; // Usuario que realizó la acción
18
19     private Long transportistaIdAnterior;
20     private Long transportistaIdNuevo;
21     private String tipoTarjetaAnterior;
22     private String tipoTarjetaNuevo;
23     private Integer cantidadAnterior;
24     private Integer cantidadNuevo;
25     private String estadoAnterior;
26     private String estadoNuevo;
27     private LocalDateTime fechaEntregaAnterior;
28     private LocalDateTime fechaEntregaNuevo;
29
30     private LocalDateTime fechaHora;
31     private String ipAddress;
32
33     @Column(length = 5000)
34     private String datosCompletosAnteriores;
35     @Column(length = 5000)
36     private String datosCompletosNuevos;
37 }
```

ejemplo de una tabla que auditoría

12 K6 - IMÁGENES DEL PROYECTO

```
y refused it."

■ TOTAL RESULTS

checks_total.....: 10740 211.421379/s
checks_succeeded...: 44.49% 4779 out of 10740
checks_failed.....: 55.50% 5961 out of 10740

X GET carga status
  0% - ✓ 0 / X 5370
X GET listar cargas
  88% - ✓ 4779 / X 591

HTTP
http_req_duration.....: avg=1.05s   min=0s     med=554.32ms max=5.26s p(90)=3.01s p(95)=
3.8s
{ expected_response:true }...: avg=911.88ms min=1.34ms med=452.54ms max=5.05s p(90)=2.62s p(95)=
3.63s
http_req_failed.....: 37.00% 5961 out of 16110
http_reqs.....: 16110 317.132068/s

EXECUTION
iteration_duration.....: avg=4.16s   min=1.06s   med=3.85s   max=7.86s p(90)=7.42s p(95)=
7.62s
iterations.....: 5370 105.710689/s
vus.....: 53 min=10 max=1000
vus_max.....: 1000 min=1000 max=1000

NETWORK
data_received.....: 7.6 MB 150 kB/s
data_sent.....: 3.4 MB 66 kB/s

running (0m50.8s), 0000/1000 VUs, 5370 complete and 0 interrupted iterations
default ✓ [=====] 0000/1000 VUs 50s

D:\Proyecto astramaco iii\backend-astramaco\k6>
```

La prueba realizada con 1,000 usuarios virtuales (VUs)


```
C:\Windows\System32\cmd.e X + v
Grafana

execution: local
script: pedido-test.js
output: -

scenarios: (100.00%) 1 scenario, 1000 max VUs, 1m20s max duration (incl. graceful stop):
* default: Up to 1000 looping VUs for 50s over 5 stages (gracefulRampDown: 30s, graceful
Stop: 30s)

TOTAL RESULTS

checks_total.....: 6033    118.580111/s
checks_succeeded....: 100.00% 6033 out of 6033
checks_failed.....: 0.00%   0 out of 6033

✓ status 200

HTTP
http_req_duration.....: avg=1.34s min=502.49µs med=1.23s max=4.34s p(90)=2.95s p(95)=3.19
s
{ expected_response:true }....: avg=1.34s min=502.49µs med=1.23s max=4.34s p(90)=2.95s p(95)=3.19
s
http_req_failed.....: 0.00%   0 out of 12066
http_reqs.....: 12066    237.160222/s

EXECUTION
iteration_duration.....: avg=3.68s min=1.05s    med=3.53s max=7.97s p(90)=6.81s p(95)=7.1s

iterations.....: 6033    118.580111/s
vus.....: 70    min=10    max=1000
vus_max.....: 1000    min=1000    max=1000

NETWORK
data_received.....: 5.7 MB 112 kB/s
data_sent.....: 2.5 MB 49 kB/s

running (0m50.9s), 0000/1000 VUs, 6033 complete and 0 interrupted iterations
default ✓ [=====] 0000/1000 VUs  50s

D:\Proyecto astramaco iii\backend-astramaco\k6>
```

El servicio de pedidos muestra un comportamiento estable y altamente confiable bajo la carga configurada:


```
C:\Windows\System32\cmd.e X + v

  \M  \K  \G
  /  /  /
 /  /  /
/  /  /

execution: local
script: transportistas-test.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 50s max duration (incl. graceful stop):
* default: 10 looping VUs for 20s (gracefulStop: 30s)

TOTAL RESULTS

checks_total.....: 570      27.916938/s
checks_succeeded...: 100.00% 570 out of 570
checks_failed.....: 0.00%   0 out of 570

✓ login responde 200
✓ token generado
✓ transportistas responde 200

HTTP
http_req_duration.....: avg=36.28ms min=2.51ms med=44.71ms max=76.07ms p(90)=70.43ms p(95)=71.65ms
{ expected_response:true }...: avg=36.28ms min=2.51ms med=44.71ms max=76.07ms p(90)=70.43ms p(95)=71.65ms
http_req_failed.....: 0.00%   0 out of 380
http_reqs.....: 380      18.611292/s

EXECUTION
iteration_duration.....: avg=1.07s   min=1.06s   med=1.07s   max=1.11s   p(90)=1.07s   p(95)=1.09s
iterations.....: 190      9.305646/s
vus.....: 10      min=10      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 180 kB  8.8 kB/s
data_sent.....: 80 kB  3.9 kB/s

running (20.4s), 00/10 VUs, 190 complete and 0 interrupted iterations
default ✓ [=====] 10 VUs  20s

D:\Proyecto astramaco iii\backend-astramaco\k6>
```

Esta prueba se ejecutó con un escenario más ligero (10 VUs) y muestra un desempeño altamente eficiente:

```
C:\Windows\System32\cmd.e X + v

  V  N  K  O
  / \ / \ / \
 /   /   /   /
/___/___/___/

execution: local
script: usuario-test.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 50s max duration (incl. graceful stop):
* default: 10 looping VUs for 20s (gracefulStop: 30s)

TOTAL RESULTS

checks_total.....: 540      26.245681/s
checks_succeeded...: 100.00% 540 out of 540
checks_failed.....: 0.00%   0 out of 540

✓ Login status 200
✓ Usuario creado exitosamente (200 o 201)
✓ Tiempo de respuesta < 500ms

HTTP
http_req_duration.....: avg=70.74ms min=57.64ms med=71.28ms max=83.3ms p(90)=76.03ms p(95)=77.73ms
{ expected_response:true }...: avg=70.74ms min=57.64ms med=71.28ms max=83.3ms p(90)=76.03ms p(95)=77.73ms
http_req_failed.....: 0.00%   0 out of 360
http_reqs.....: 360      17.497121/s

EXECUTION
iteration_duration.....: avg=1.14s   min=1.12s   med=1.14s   max=1.16s   p(90)=1.15s   p(95)=1.15s
iterations.....: 180      8.74856/s
vus.....: 10      min=10      max=10
vus_max.....: 10      min=10      max=10

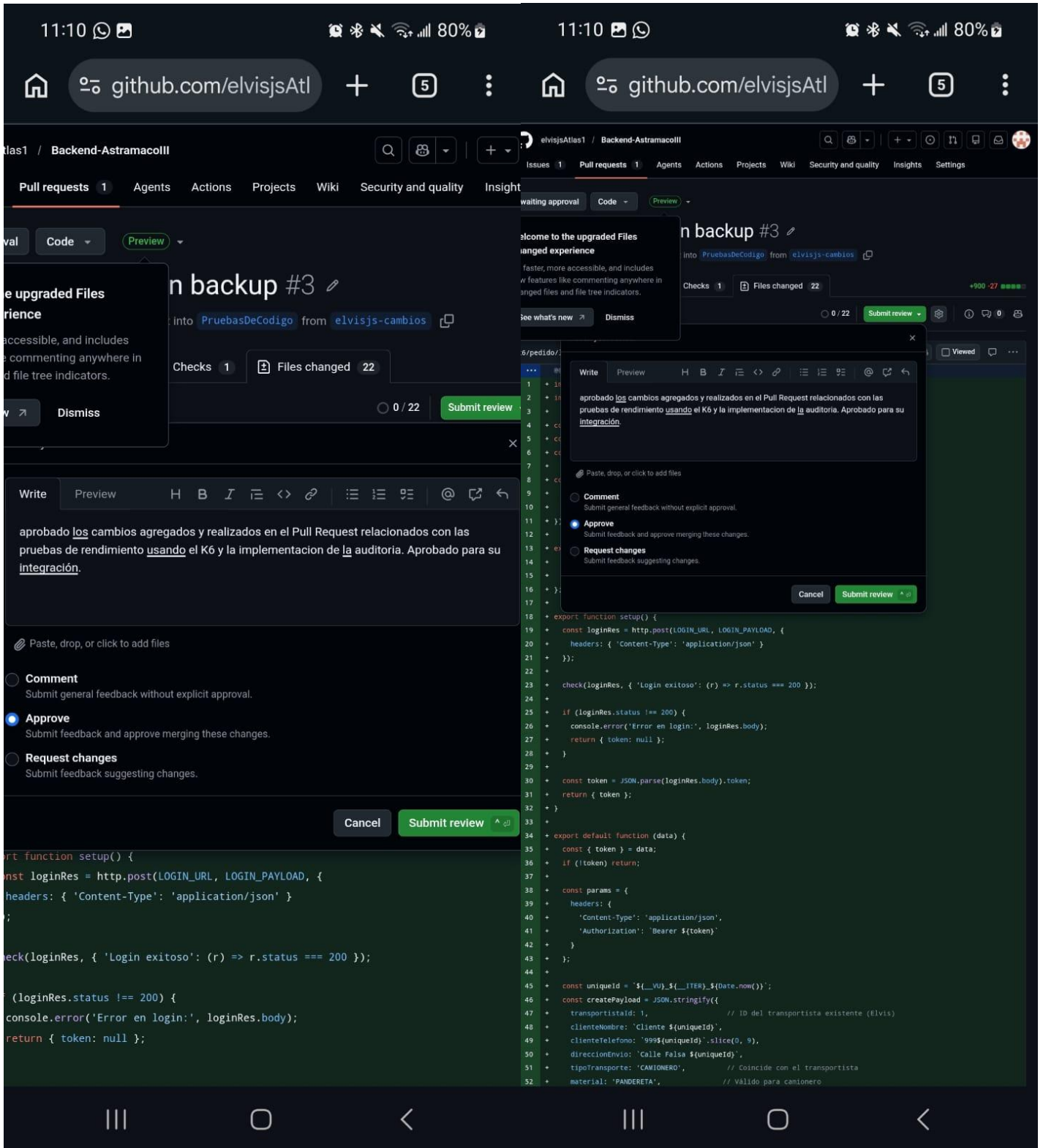
NETWORK
data_received.....: 221 kB  11 kB/s
data_sent.....: 98 kB  4.7 kB/s

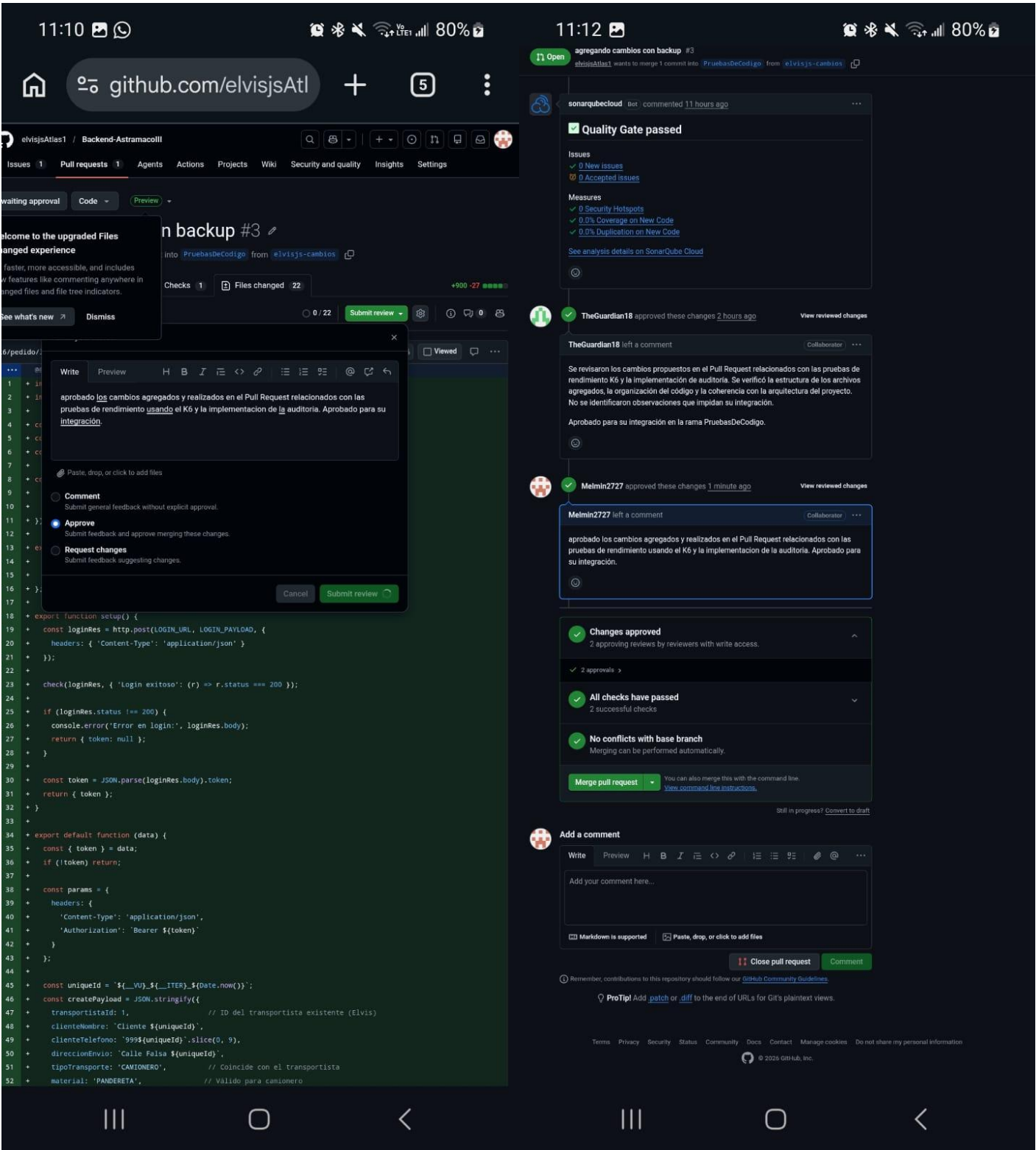
running (20.6s), 00/10 VUs, 180 complete and 0 interrupted iterations
default ✓ [=====] 10 VUs  20s

D:\Proyecto astramaco iii\backend-astramaco\k6>
```

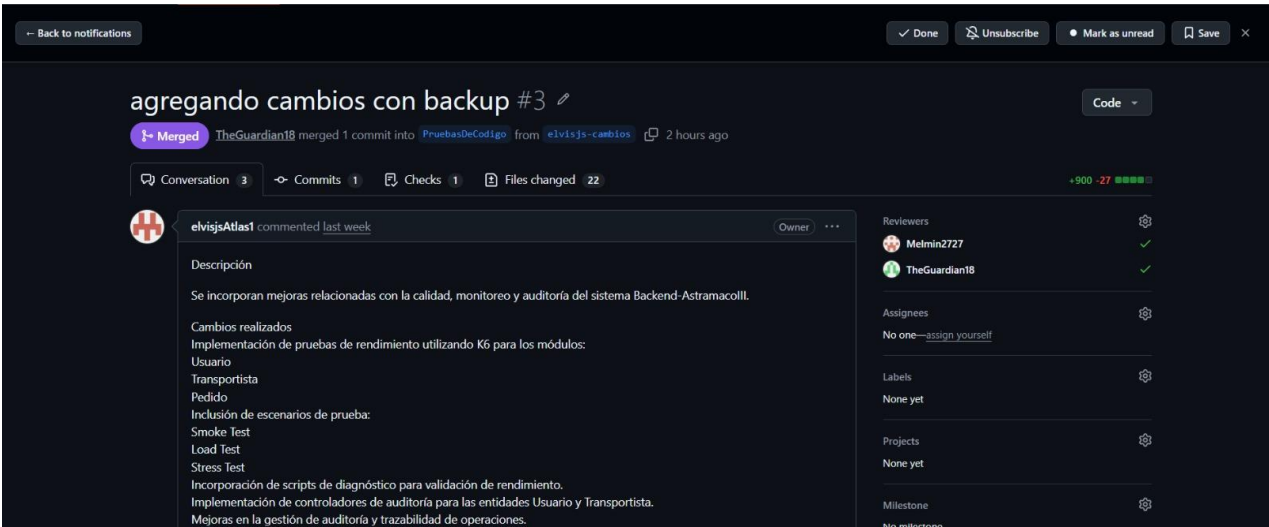
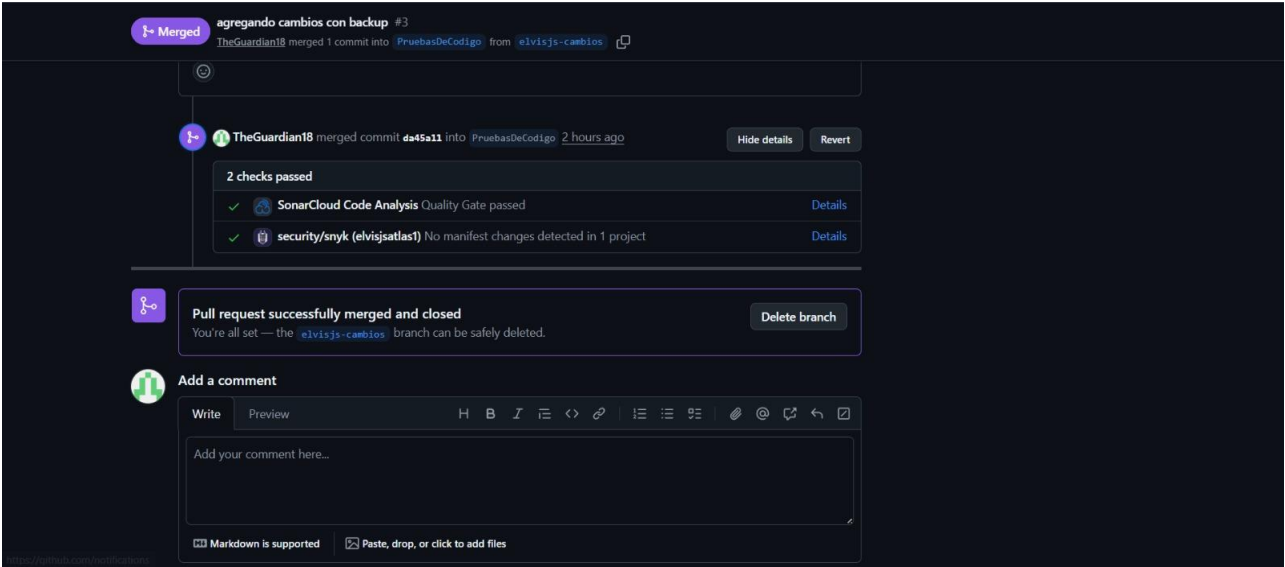
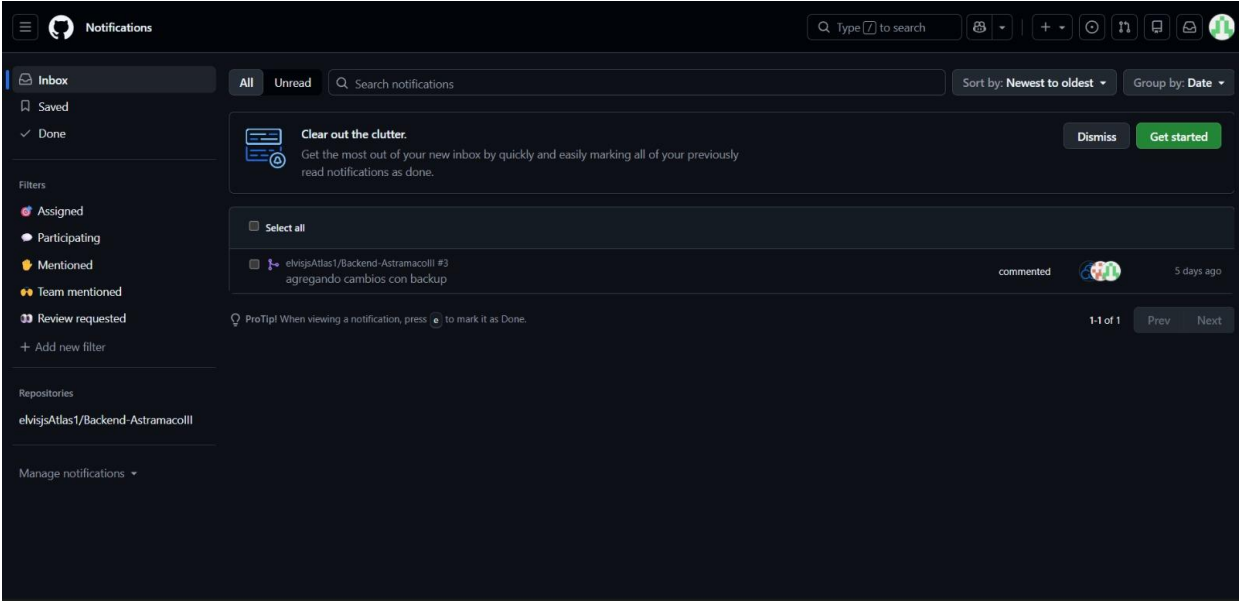
El módulo de usuarios demuestra un rendimiento sólido y altamente eficiente bajo la carga configurada (10 VUs):

REVISORES



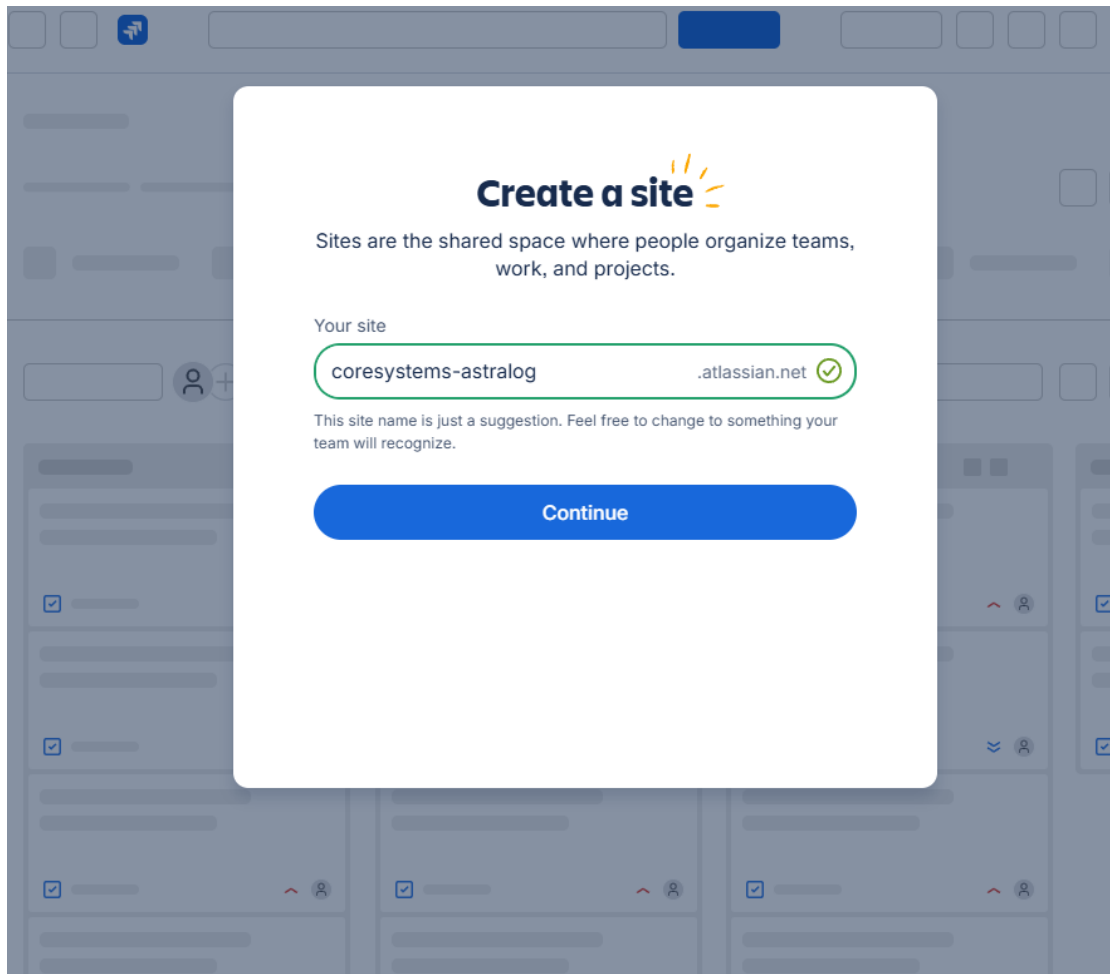


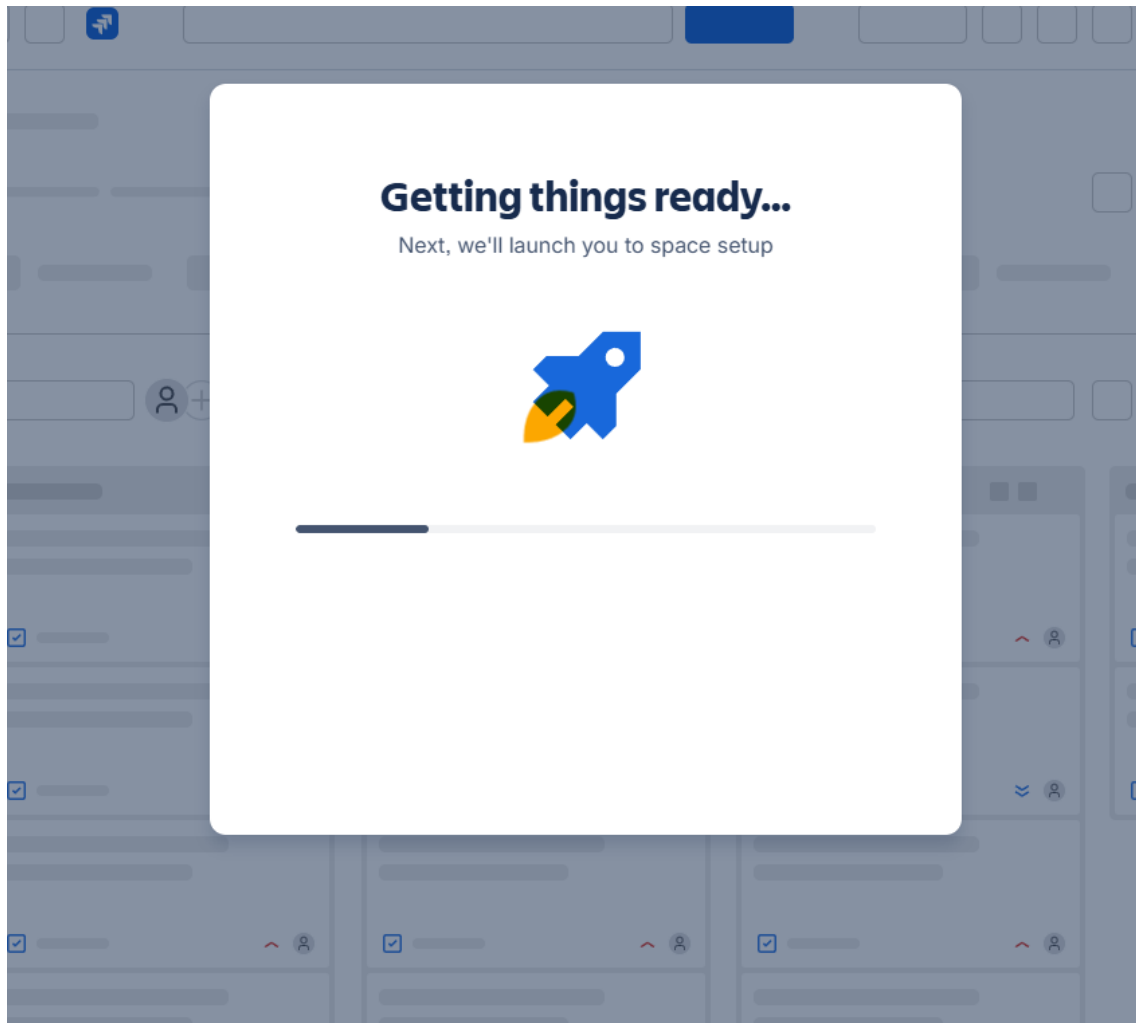
PRIMER REVISOR: La consolidación del código se ha llevado a cabo bajo un estricto proceso de revisión por pares, garantizando la integridad y calidad del sistema Backend-Astramacolli.



SEGUNDO REVISOR: La consolidación del código se ha llevado a cabo bajo un estricto proceso de revisión por pares, garantizando la integridad y calidad del sistema Backend-Astramacolli.

14 CMMI+Scrum utilizando JIRA (las 3 áreas TS, VER, IP)





What kind of work do you do?

This helps us personalize your Jira experience

 Software development	 Product management
 Marketing	 Design
 Project management	 Operations
 IT support	 Human resources
 Customer service	 Legal
 Finance	 Sales
 Data science	 Other

How does your team plan to use Jira?

☒ Prioritize work

☒ Track bugs

☒ Run sprints

☐ Manage tasks

☐ Map work dependencies

☒ Work in scrum

Back

Continue

Name your space

This is where you'll help your team track progress, stay organized, and manage tasks.

Name your space

ASTRAMACO III - AstraLog

Example space names:

My Software Team

Team Astro

Front-End Dev

QA Testing

Bug Tracking

Get started



What types of work do you need?

These form the building blocks of your space.

☒ **Task**
A small piece of work.

☒ **Story**
A requirement expressed from the user's perspective.

☒ **Feature**
A broad piece of functionality.

☒ **Request**
An ask for assistance.

☒ **Bug**
A problem that needs fixing.

Don't worry, you can change these later.

Next



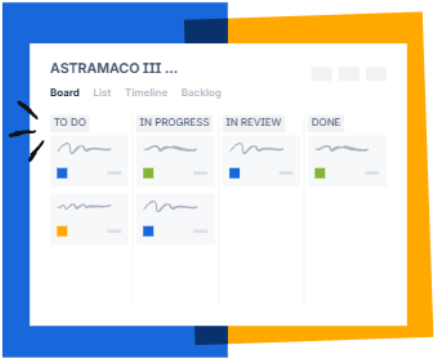
How do you track work?

As you complete work, it moves through these statuses.

To Do	X
In Progress	X
In Review	X
Done	X

+ Add status

Next



Setting up your space

Hold tight while we finish setting things up...

Share via link

<https://coresystems-as>

Copy link

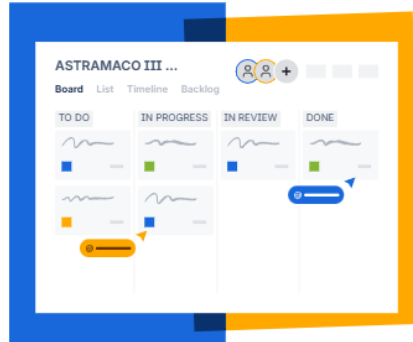
Invite via email

chat02jselvis@gmail.com

yuniorbenitoquispequispe@gmail.com

add more people...

This site is protected by reCAPTCHA and the Google [Privacy Policy](#) and [Terms of Service](#) apply.



Do this later



Jira

Para ti

Recientes

Marcados como fav...

Aplicaciones

Planes

Espacio

Recientes

ASTRAMACO III - Ast...

Más espacios

Filtros

Paneles

Equipos

Personalizar barra lateral

Buscar

+ Crear

Ver los planes

Espacio

ASTRAMACO III - AstraLog

Resumen

Backlog

Tablero

Código

Cronograma

Documentos

Formularios

Buscar tablero

Filtro

Completar sprint

Grupo

TO DO

Tarea 1

5 jun 2026

SCRUM-1

+ Crear

IN PROGRESS

Tarea 2

10 jun 2026

SCRUM-2

IN REVIEW

DONE

Buscar

+ Crear

Ver los planes

Resumen

Ba

ASTRAMACO

Sprint 1: Análisis y Diseño

SCRUM-1 Ta

SCRUM-2 Ta

+ Crear

Backlog (1 a

SCRUM-3 Ta

+ Crear

Inicio sprint

es | Estimación

Crear sp

Editar sprint: Sprint 1: Análisis y Diseño

Los campos obligatorios están marcados con un asterisco *

Nombre del sprint *

Sprint 1: Análisis y Diseño

Duración

personalizado

Fecha de inicio

30/3/2026 12:00

Fecha de finalización

8/5/2026 12:00

Meta del sprint

Cancelar

Actualizar

Ver configuración



Panel de Epic

E



Sprints vacíos



Densidad



Predeterminado



Compacto



Campos

Tipo de actividad



Clave de actividad



Epic



Fecha de vencimiento



Estado



Persona asignada





SCRUM-11



1



IP - Integrated Project Management



Tareas por hacer ▾



⌘ Mejorar el tipo de actividad Epic

▼ Descripción

Planificación, monitoreo y control del proyecto.

▼ Actividades secundarias



100 % completado

Actividad	P...	P...	Estado
SCRUM-12	IP-01...	=	FINALIZ

Actividades vinculadas

Añadir actividad vinculada

Contenido de Confluence ⓘ



Plan del proyecto

PROBAR PLANTILLA



Detalles



Persona asignada



☒ Actividad de Jira



⚡ SCRUM-11



👁 1



IP - Integrated Project Management



Tareas por hacer ▾



⌘ Mejorar el tipo de actividad Epic

▾ Descripción

Planificación, monitoreo y control del proyecto.

▾ Actividades secundarias



66 % completado

Actividad		P...	P...	Estado
 SCRUM-12	IP-01...	=		FINALIZ
 SCRUM-13	IP-0...	=		EN REVIS
 SCRUM-14	IP-0...	=		FINALIZ

Actividades vinculadas

Añadir actividad vinculada

Contenido de Confluence



Plan del proyecto

PROBAR PLANTILLA



✓ Actividad de Jira

↗

×

🔗 SCRUM-11 / 📌 SCRUM-12

🔒

👁

🔗

⋮

IP-01: Planificación del Proyecto AstraLog

+

⋮

Finalizada ▾

✓ Listo

⚡

⚙ Mejorar el tipo de actividad Historia

▼ Descripción

Fase inicial del proyecto centrada en el análisis de negocio y definición de requisitos bajo estándares de ingeniería.

▼ Subtareas

⋮ 📌 +

🔗 Crea las actividades sugeridas

Sugerir

50 % completado

Actividad	Pri...	Per...	Estado
🔗 SCRUM-15 Análisis de negocio	== M.	C c	FINALIZADA ▾
🔗 SCRUM-16 Definición de requisitos	== M.	AV A	FINALIZADA ▾
🔗 SCRUM-17 Planificación de sprints	== M.	YQ Y	EN REVISIÓN ▾
🔗 SCRUM-18 Identificación de riesgos	== M.	C c	EN REVISIÓN ▾

Pon un nombre a esta subtask

🔗 Subtask ▾

↩

🔍 Elegir una existente

Cancelar

Actividades vinculadas

IP-02: Seguimiento del Proyecto

+

⋮

En revisión ▾ ⚡ ⚡ Mejorar el tipo de actividad Historia

Descripción

Fase continua del proyecto centrada en el monitoreo del progreso, control de calidad del código y mitigación de riesgos durante todo el ciclo de desarrollo.

Subtareas

⋮ 📄 +

⚡ Crea las actividades sugeridas

Sugerir

75 % completado

Actividad	Pri...	Per...	Estado
SCRUM-19 Seguimiento Sprint 1	⚡ M.	C c	FINALIZADA ▾
SCRUM-20 Seguimiento Sprint 2	⚡ M.	C c	FINALIZADA ▾
SCRUM-21 Seguimiento Sprint 3	⚡ M.	AV A	FINALIZADA ▾
SCRUM-22 Seguimiento Sprint 4	⚡ M.	YQ Y	EN REVISIÓN ▾

Pon un nombre a esta subtask

🔗 Subtask ▾

↩

Elegir una existente

Cancelar

Actividades vinculadas



Actividad de Jira

SCRUM-11 / SCRUM-14

IP-03: Gestión de Riesgos

Finalizada Listo Mejorar el tipo de actividad Historia

Descripción

Fase enfocada en la identificación temprana, análisis y mitigación de posibles riesgos técnicos y operativos para asegurar la estabilidad del proyecto.

Subtareas

Crea las actividades sugeridas

66 % completado

Actividad	Pri...	Per...	Estado
SCRUM-23 Riesgo de fallas GPS	M.	C	FINALIZADA
SCRUM-24 Riesgo de retrasos Backend	M.	C	FINALIZADA
SCRUM-25 Riesgo de ausencia de miembros	M.	AV	EN REVISIÓN

Pon un nombre a esta subtask

Elegir una existente

Cancelar

Actividades vinculadas

Añadir actividad vinculada

TS:

Historias para el Sprint 1: Análisis y Diseño

1. TS-01: Análisis de Negocio y Flujo de Ventas

- **Sprint:** Sprint 1: Análisis y Diseño
- **Subtareas:** Identificar actores, Identificar procesos, Analizar flujo actual

2. TS-02: Definición de Requisitos

- **Sprint:** Sprint 1: Análisis y Diseño
- **Subtareas:** Requisitos funcionales, Requisitos no funcionales, Historias de usuario

3. TS-03: Modelado de Historias de Usuario

- **Sprint:** Sprint 1: Análisis y Diseño
- **Subtareas:** HU01 Registro de Ventas, HU02 Asignación Equitativa, HU03 Hoja de Ruta, HU04 Evidencia Fotográfica, HU05 Tracking, HU06 Notificación Cliente

4. TS-04: Arquitectura de Microservicios y Base de Datos

- **Sprint:** Sprint 1: Análisis y Diseño
- **Subtareas:** Diseño Backend, Diseño BD, Diseño API REST

5. TS-05: Diseño de Interfaces

- **Sprint:** Sprint 1: Análisis y Diseño
- **Subtareas:** Dashboard Administrador, App Transportista, Pantalla Ventas, Pantalla Tracking

Historias para el Sprint 2: Desarrollo Backend

6. TS-06: Configuración Backend Spring Boot

- **Sprint:** Sprint 2: Desarrollo Backend
- **Subtareas:** Spring Boot, Lombok, JPA, JWT

7. TS-07: Implementar Algoritmo de Equidad

- **Sprint:** Sprint 2: Desarrollo Backend
- **Subtareas:** Asignación por prioridad, Validación de carga semanal, Rotación de servicios

8. TS-09: Módulo de Costos *(Nota: Refleja tu cálculo de costos de transporte)*

- **Sprint:** Sprint 2: Desarrollo Backend
- **Subtareas:** Cálculo distancia, Cálculo por piso, Costo total

Historias para el Sprint 3: Desarrollo Frontend y App Móvil

9. TS-08: Desarrollo de App Móvil

- **Sprint:** Sprint 3: Desarrollo Frontend y App Móvil
- **Subtareas:** Recepción hoja de ruta, Visualización de pedidos, Cambio de estado

El screenshot muestra la interfaz de AstraLog con el siguiente contenido:

- Encabezado:** Espacio, ASTRAMACO III - AstraLog, pestañas (Resumen, Backlog, Tablero, Código, Cronograma, Documentos, Formularios).
- Barra de búsqueda:** Q. Buscar en el ba..., Filtro.
- Sprint 1: Análisis y Diseño** (30 mar - 8 may, 7 actividades):

SCRUM-12	IP-01: Planificación del Proyecto AstraLog	IP - INTEGRATED PR...	EN CURSO	31 jul
SCRUM-14	IP-03: Gestión de Riesgos	IP - INTEGRATED PR...	EN CURSO	31 jul
SCRUM-26	TS-01: Análisis de Negocio y Flujo de Ventas	TS - TECHNICAL SOL...	EN REVISIÓN	
SCRUM-30	TS-02: Definición de Requisitos	TS - TECHNICAL SOL...	EN REVISIÓN	
SCRUM-34	TS-03: Modelado de Historias de Usuario	TS - TECHNICAL SOL...	EN REVISIÓN	
SCRUM-41	TS-04: Arquitectura de Microservicios y Base de Datos	TS - TECHNICAL SOL...	EN REVISIÓN	
SCRUM-45	TS-05: Diseño de Interfaces	TS - TECHNICAL SOL...	EN REVISIÓN	
- Sprint 2: Desarrollo Backend** (4 may - 23 may, 3 actividades):

SCRUM-50	TS-06: Configuración Backend Spring Boot	TS - TECHNICAL SOL...	EN REVISIÓN	
SCRUM-55	TS-07: Implementar Algoritmo de Equidad	TS - TECHNICAL SOL...	EN REVISIÓN	
SCRUM-59	TS-09: Módulo de Costos	TS - TECHNICAL SOL...	EN REVISIÓN	
- Desarrollo Frontend y AppMóvil** (4 may - 23 may, 1 actividad):

SCRUM-63	TS-08: Desarrollo de App Móvil	TS - TECHNICAL SOL...	EN REVISIÓN	
----------	--------------------------------	-----------------------	-------------	--

VER:

📁 Historias para el Sprint 2: Desarrollo Backend

1. VER-01: Pruebas Unitarias

- **Sprint:** Sprint 2: Desarrollo Backend
- **Subtareas:** Prueba algoritmo, Prueba costos, Prueba autenticación

📁 Historias para el Sprint 3: Desarrollo Frontend y App Móvil

2. VER-02: Pruebas de Integración

- **Sprint:** Sprint 3: Desarrollo Frontend y App Móvil
- **Subtareas:** Frontend + Backend, Backend + BD, App + API

📁 Historias para el Sprint 4: Pruebas, Calidad y Despliegue

3. VER-03: Análisis SonarQube

- **Sprint:** Sprint 4: Pruebas, Calidad y Despliegue
- **Subtareas:** Code Smells, Bugs, Security Hotspots

4. VER-04: Corrección de Vulnerabilidades

- **Sprint:** Sprint 4: Pruebas, Calidad y Despliegue

- **Subtareas:** Eliminar credenciales hardcodedas, Variables de entorno, Nueva validación Sonar

5. VER-05: Validación de Evidencia Fotográfica

- **Sprint:** Sprint 4: Pruebas, Calidad y Despliegue
- **Subtareas:** Carga imagen, Validación GPS, Registro entrega

El espacio de trabajo de Jira para el proyecto 'ASTRAMACO III - AstraLog' muestra una estructura de sprints y tareas. El sidebar izquierdo permite navegar por las categorías de Epic: 'Sin epic', 'TS - Technical Solution', 'VER - Verification' y 'IP - Integrated Project Management'. El panel principal está dividido en tres secciones de sprints:

- Sprint 2: Desarrollo Backend** (4 May - 23 May): Incluye tareas como SCRUM-50 (TS-06: Configuración Backend Spring Boot), SCRUM-55 (TS-07: Implementar Algoritmo de Equidad), SCRUM-59 (TS-08: Módulo de Costos) y SCRUM-67 (VER-01: Pruebas Unitarias).
- Desarrollo Frontend y AppMóvil** (4 May - 23 May): Incluye tareas como SCRUM-63 (TS-08: Desarrollo de App Móvil) y SCRUM-71 (VER-02: Pruebas de Integración).
- Pruebas, Calidad y Despliegue** (9 May - 31 May): Incluye tareas como SCRUM-75 (VER-03: Análisis SonarQube), SCRUM-79 (VER-04: Corrección de Vulnerabilidades) y SCRUM-83 (VER-05: Validación de Evidencia Fotográfica).

En la parte inferior, se muestra una actividad específica 'SCRUM-86' con un botón para 'Copiar enlace'.

Iniciamos el Sprint 1: Analisis y diseño

Buscar

Crear

Ver los planes

Espacio

ASTRAMACO III - AstraLog

Resumen Backlog Tablero Código

Buscar en el ba...

Filtro

Epic

Sin epic

TS - Technical Solution

VER - Verification

IP - Integrated Project Management

Crear epic

Sprint

SCRUM-12 IP-01: Pla...

SCRUM-14 IP-03: Ge...

SCRUM-26 TS-01: An...

SCRUM-30 TS-02: De...

SCRUM-41 TS-04: Ar...

SCRUM-34 TS-03: Mo...

SCRUM-45 TS-05: Dis...

Crear

Sprint 1: Análisis y Diseño

31 may - 14 jun (7 actividades)

0 10 25 Completar sprint

Completar el análisis de negocio, la definición de requisitos y el diseño de la arquitectura base de AstraLog.

SCRUM-12 IP-01: Pla...

SCRUM-14 IP-03: Ge...

SCRUM-26 TS-01: An...

SCRUM-30 TS-02: De...

SCRUM-41 TS-04: Ar...

SCRUM-34 TS-03: Mo...

SCRUM-45 TS-05: Dis...

Crear

Sprint 2: Desarrollo Backend

4 abr - 18 abr (4 actividades)

0 10 10 Completar sprint

tener arquitectura sprint boot monolito que soporte una capacidad de hasta 50 usuario al mismo tiempo.

SCRUM-55 TS-07: Im...

SCRUM-59 TS-09: Mó...

SCRUM-60 TS-06: Co...

SCRUM-67 VER-01: Pr...

Crear

Iniciar sprint

7 actividades se incluirán en este sprint.

Los campos obligatorios están marcados con un asterisco *

Nombre del sprint *

Sprint 1: Análisis y Diseño

Duración *

2 semanas

Fecha de inicio *

31/5/2026 16:23

Fecha de inicio prevista: 30/3/2026, 12:00

La fecha de inicio de un sprint afecta a la velocidad y al alcance de los informes. [Obtén más información.](#)

Fecha de finalización *

14/6/2026 18:23

Meta del sprint

Completar el análisis de negocio, la definición de requisitos y el diseño de la arquitectura base de AstraLog.

Cancelar Iniciar

Actividad "SCRUM-86"

Copiar enlace

Espacio

ASTRAMACO III - AstraLog

Resumen Backlog Tablero Código Cronograma Documentos Formularios

Buscar en el ba...

Filtro

Epic

Sin epic

TS - Technical Solution

VER - Verification

IP - Integrated Project Management

Crear epic

Sprint 1: Análisis y Diseño

31 may - 14 jun (7 actividades)

0 10 25 Completar sprint

Completar el análisis de negocio, la definición de requisitos y el diseño de la arquitectura base de AstraLog.

SCRUM-12 IP-01: Pla...

SCRUM-14 IP-03: Ge...

SCRUM-26 TS-01: An...

SCRUM-30 TS-02: De...

SCRUM-41 TS-04: Ar...

SCRUM-34 TS-03: Mo...

SCRUM-45 TS-05: Dis...

Crear

Sprint 2: Desarrollo Backend

4 abr - 18 abr (4 actividades)

0 10 10 Completar sprint

tener arquitectura sprint boot monolito que soporte una capacidad de hasta 50 usuario al mismo tiempo.

SCRUM-55 TS-07: Im...

SCRUM-59 TS-09: Mó...

SCRUM-60 TS-06: Co...

SCRUM-67 VER-01: Pr...

Crear

Espacio

ASTRAMACO III - AstraLog

Resumen Backlog Tablero Código Cronograma Documentos Formularios

Buscar en el ba...

Filtro

Epic

Sin epic

TS - Technical Solution

VER - Verification

IP - Integrated Project Management

Crear epic

Sprint 1: Análisis y Diseño

31 may - 14 jun (7 actividades)

0 10 25 Completar sprint

Completar el análisis de negocio, la definición de requisitos y el diseño de la arquitectura base de AstraLog.

SCRUM-12 IP-01: Pla...

SCRUM-14 IP-03: Ge...

SCRUM-26 TS-01: An...

SCRUM-30 TS-02: De...

SCRUM-41 TS-04: Ar...

SCRUM-34 TS-03: Mo...

SCRUM-45 TS-05: Dis...

Crear

Sprint 2: Desarrollo Backend

4 abr - 18 abr (4 actividades)

0 10 10 Completar sprint

tener arquitectura sprint boot monolito que soporte una capacidad de hasta 50 usuario al mismo tiempo.

SCRUM-55 TS-07: Im...

SCRUM-59 TS-09: Mó...

SCRUM-60 TS-06: Co...

SCRUM-67 VER-01: Pr...

Crear

Actividad de Jira

SCRUM-11 / SCRUM-12

IP-01: Planificación del Proyecto AstraLog

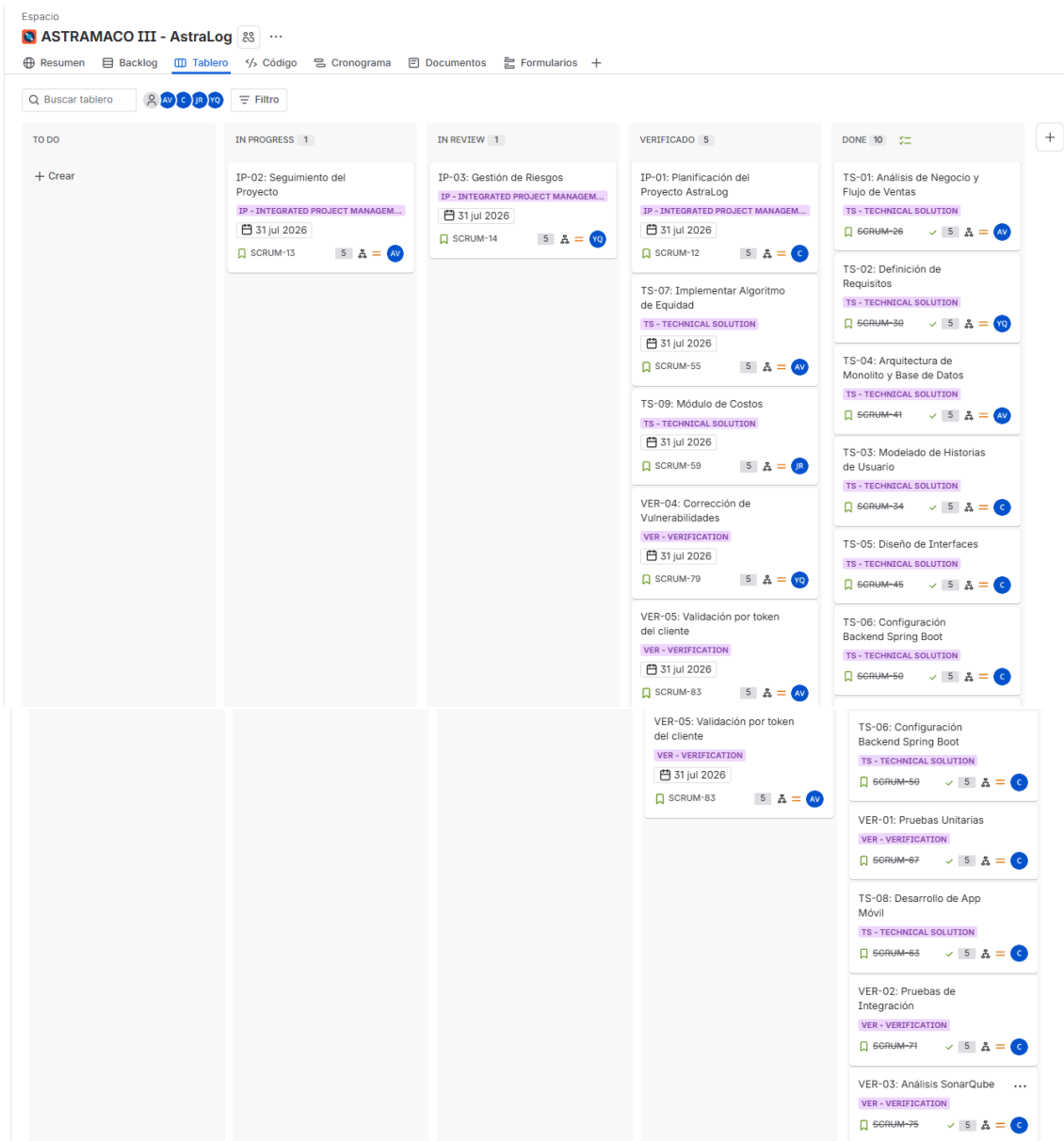
VERIFICADO

Mejorar el tipo de actividad Historia

Descripción

Descripción: Como equipo de gestión del proyecto AstraLog, se requiere realizar la planificación integral del proyecto con el fin de definir el alcance, objetivos, recursos, cronograma, riesgos y estrategia de ejecución bajo el marco Scrum. Esta actividad permitirá organizar de manera eficiente las fases de análisis, desarrollo, pruebas y despliegue, asegurando el cumplimiento de los requerimientos de ASTRAMACO III y el logro de los objetivos establecidos para el sistema. **Criterios de Aceptación** [Definition of Done - CMMI]:

- [TS - Solución Técnica]:
 - Inicializar el proyecto usando Spring Initializr con Java 21, Spring Boot 3.5.14 y Maven.
 - Incluir dependencias base en el 'pom.xml': Spring Web, DevTools y Lombok.
 - Definir una estructura de paquetes limpia (config, controller, service, repository, model).
 - El archivo 'application.yml' o 'properties' debe estar modularizado y sin credenciales quemadas.
- [VER - Verificación]:
 - El proyecto debe compilar limpiamente sin advertencias críticas ('maven clean compile' exitoso).
 - Verificar que la aplicación levante localmente en el puerto asignado sin lanzar excepciones.
 - Configurar la clase de prueba base '@SpringBootTest' para comprobar la carga del contexto.



DASHBOARD FINAL

Crear Dashboard:

ASTRALOG - CMMI + SCRUM

Agregar:

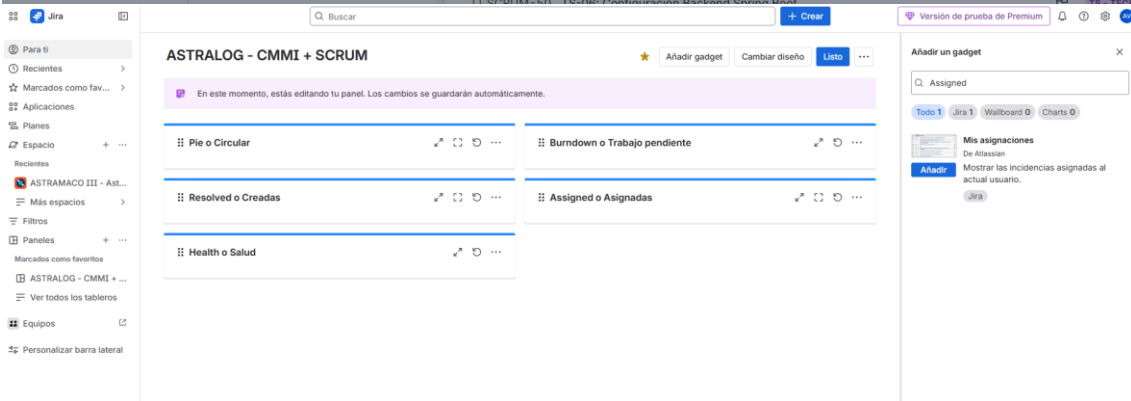
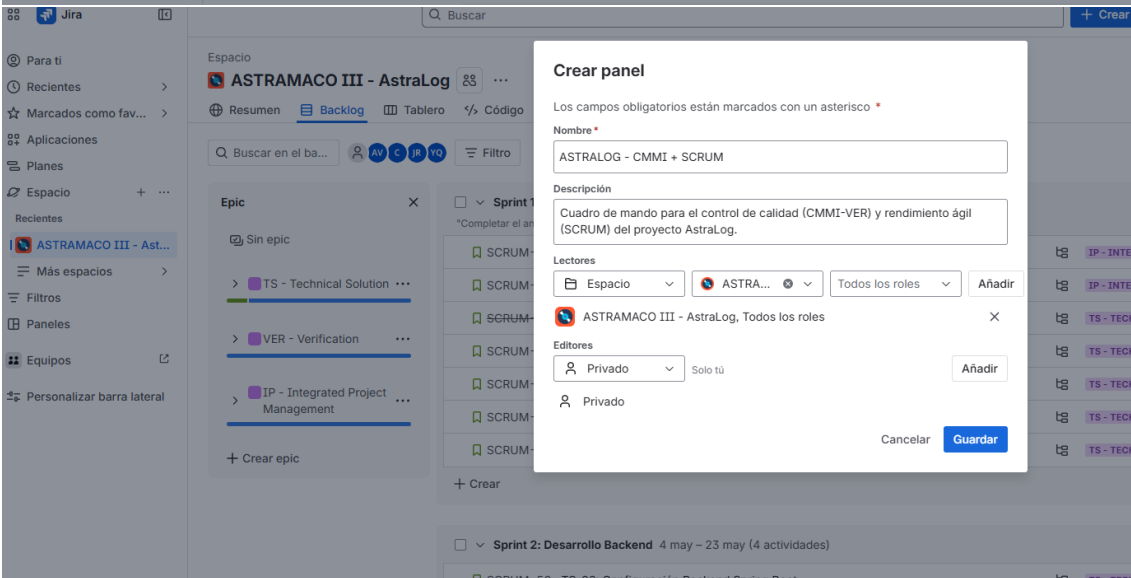
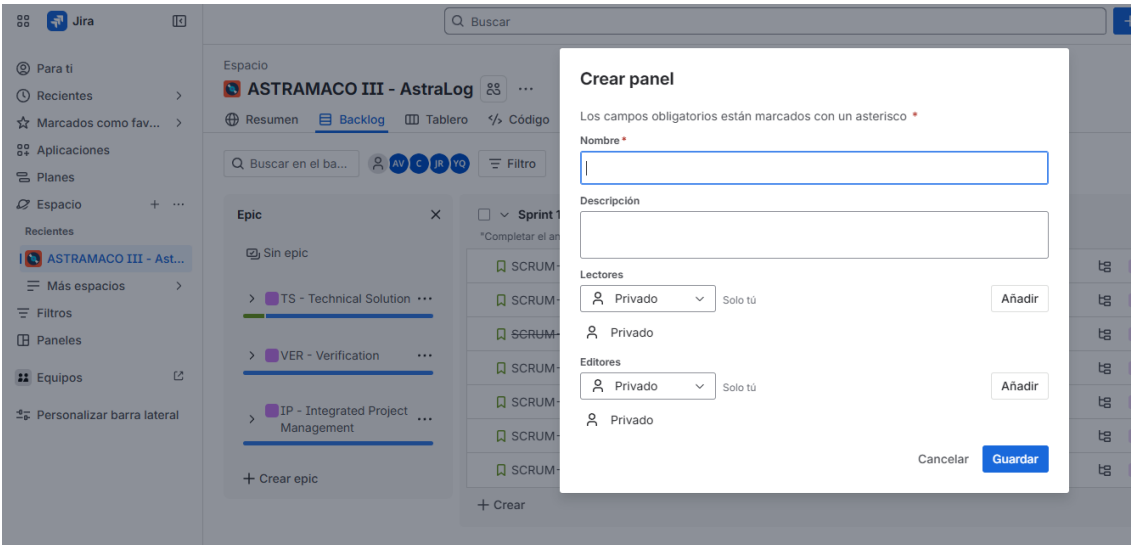
Sprint Burndown

Sprint Health

Pie Chart

Created vs Resolved

Assigned Issues



TERMINANO:

TS

- ✓ Arquitectura
- ✓ Historias de Usuario
- ✓ Diseño BD
- ✓ Algoritmo de Equidad
- ✓ App Móvil

VER

- ✓ JUnit
- ✓ Integración
- ✓ SonarQube
- ✓ Corrección de Vulnerabilidades

IP

- ✓ Backlog
- ✓ Sprints
- ✓ Gestión de Riesgos
- ✓ Dashboard
- ✓ Seguimiento

15 REFERENCIAS

- [1] C4-PlantUML. (s.f.). C4 Model for PlantUML. Recuperado de:
<https://github.com/plantuml-stdlib/C4-PlantUML>
- [2] Brown, S. (2018). The C4 model for visualising software architecture.
Recuperado de: <https://c4model.com>
- [3] Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly Media.
- [4] Richardson, C. (2019). Microservices Patterns. Manning Publications.
- [5] Evans, E. (2003). Domain-Driven Design. Addison-Wesley.
- [6] Martin, R. C. (2003). Agile Software Development. Prentice Hall.

16 ANEXOS

16.1 ANEXO A: CÓDIGO PLANTUML – DIAGRAMA DE CONTEXTO

(C4 NIVEL 1)

```
@startuml
!includeurl https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml

Person(admin, "Administrador Logístico", "Registra pedidos y gestiona la flota (web)")
Person(chofer, "Transportista (Chofer)", "Visualiza rutas y sube evidencias (móvil)")
Person(dueno, "Dueño / Gerencia", "Supervisa comisiones y KPIs de rendimiento (web/móvil)")
Person_Ext(cliente, "Cliente", "Solicita servicios de flete de agregados")

System(sys, "AstraLog", "Sistema de gestión de transporte, tarifas y asignación equitativa")

Rel(admin, sys, "Gestiona pedidos y asigna viajes")
Rel(chofer, sys, "Recibe hojas de ruta y sube evidencias de entrega")
Rel(dueno, sys, "Audita comisiones y supervisa el negocio")
Rel(cliente, sys, "Recibe cotización dinámica automatizada")
@enduml
```

16.2 ANEXO B: CÓDIGO PLANTUML – MOLITO - VENTAS

(C4 NIVEL 2)

```
@startuml
!includeurl https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml
!includeurl https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml

Person(admin, "Administrador Logístico")
Person(chofer, "Transportista (Chofer)")
Person(dueno, "Dueño / Gerencia")

System_Boundary(sys_boundary, "Sistema AstraLog (Monolito)") {
    Container(web, "Aplicación Web", "Frontend", "Panel de administración")
    Container(mobile, "Aplicación Móvil", "Frontend", "App de choferes")

    Container(monolito, "Núcleo AstraLog", "Java Spring Boot", "Proceso único") {
```

```

        Component(auth, "Módulo de Autenticación")
        Component(pedidos, "Módulo de Pedidos")
        Component(flota, "Módulo de Inventario")
        Component(auditoria, "Módulo de Auditoría")
    }

    ContainerDb(db, "Base de Datos Unificada", "RDBMS",
"Almacenamiento central")
}

Rel(admin, web, "Usa")
Rel(chofer, mobile, "Usa")
Rel(web, monolito, "Llama métodos")
Rel(mobile, monolito, "Llama métodos")
Rel(monolito, db, "Lee/Escribe")

@enduml

```

16.3 ANEXO C: CÓDIGO PLANTUML – MONOLITO DE AUTENTICACIÓN (C4 NIVEL 3)

```

@startuml
!includeurl https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml

' Entrada principal de la aplicación web/móvil
Container(web_mobile, "Frontend", "App Web/Móvil", "Acceso del usuario")
ContainerDb(db_unificada, "Base de Datos Unificada", "RDBMS", "Almacenamiento global")

System_Boundary(monolito_auth, "Núcleo AstraLog (Módulo de Autenticación)") {
    Component(auth_controller, "Auth Controller", "REST Controller",
"Endpoints internos de login y validación")
    Component(auth_service, "Auth Service", "Service Component", "Lógica de validación de credenciales")
    Component jwt_provider, "JWT Provider", "Seguridad", "Generación y validación de tokens JWT")
    Component(auth_repository, "Auth Repository", "Repository", "Acceso a datos de credenciales")
}

' Relaciones internas (Inyección de dependencias)
Rel(web_mobile, auth_controller, "Envía credenciales")
Rel(auth_controller, auth_service, "Invoca (Lógica interna)")
Rel(auth_service, jwt_provider, "Solicita firma/validación")
Rel(auth_service, auth_repository, "Consulta credenciales")
Rel(auth_repository, db_unificada, "Lee/Escribe (esquema auth)")
@enduml

```

16.4 ANEXO D: CÓDIGO PLANTUML – MONOLITO DE USUARIOS

(C4 NIVEL 3)

```
@startuml
!includeurl https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml

' Entrada principal de la aplicación web/móvil
Container(web_mobile, "Frontend", "App Web/Móvil", "Acceso del usuario")
ContainerDb(db_unificada, "Base de Datos Unificada", "RDBMS", "Almacenamiento global")

System_Boundary(monolito_auth, "Núcleo AstraLog (Módulo de Autenticación)") {
    Component(auth_controller, "Auth Controller", "REST Controller", "Endpoints internos de login y validación")
    Component(auth_service, "Auth Service", "Service Component", "Lógica de validación de credenciales")
    Component(json_provider, "JWT Provider", "Seguridad", "Generación y validación de tokens JWT")
    Component(auth_repository, "Auth Repository", "Repository", "Acceso a datos de credenciales")
}

' Relaciones internas (Inyección de dependencias)
Rel(web_mobile, auth_controller, "Envía credenciales")
Rel(auth_controller, auth_service, "Invoca (Lógica interna)")
Rel(auth_service, json_provider, "Solicita firma/validación")
Rel(auth_service, auth_repository, "Consulta credenciales")
Rel(auth_repository, db_unificada, "Lee/Escribe (esquema auth)")
@enduml
```

16.5 ANEXO E: CÓDIGO PLANTUML – AUDITORÍA (C4 NIVEL 3)

```
@startuml
!includeurl https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml

Container(web_app, "Aplicación Web/Móvil", "Frontend", "Interfaz de usuario")
ContainerDb(db_unificada, "Base de Datos Unificada", "RDBMS", "Almacenamiento global")

System_Boundary(monolito_audit, "Núcleo AstraLog (Módulo de Auditoría)") {
    Component(audit_controller, "Auditoría Controller", "REST Controller", "Endpoints para consulta de logs")
    Component(audit_service, "Auditoría Service", "Service Component", "Procesa y filtra los logs transaccionales")
    Component(json_logger, "JSON Event Logger", "Componente", "Serializa estados a JSON")
    Component(audit_repository, "Auditoría Repository", "Repository", "Registro indeleble de marcas de auditoría")
}
```

```

}

' Relaciones internas del monolito
Rel(web_app, audit_controller, "Consulta logs (HTTP)")
Rel(audit_controller, audit_service, "Invoca búsqueda")
Rel(audit_service, json_logger, "Solicita formateo")
Rel(audit_service, audit_repository, "Persiste el registro")
Rel(audit_repository, db_unificada, "Lee/Escribe (esquema audit)")
@enduml

```

16.6 ANEXO F: CÓDIGO PLANTUML – Representación conceptual de la arquitectura de monolito (C4 NIVEL 3)

```

@startuml
skinparam componentStyle uml2

package "Capa de Presentación (Clientes)" {
    [Aplicación Web (Admin/Gerencia)] as web
    [Aplicación Móvil (Choferes)] as mobile
}

package "Núcleo AstraLog (Monolito Modular)" {
    [Módulo Autenticación] as auth
    [Módulo Usuarios] as usuarios
    [Módulo Pedidos] as pedidos
    [Módulo Inventario/Flota] as flota
    [Módulo Auditoría] as auditoria
}

package "Capa de Persistencia Unificada" {
    database "Base de Datos AstraLog" as db_main
}

' Flujos de comunicación
web --> auth : HTTPS / REST (Auth)
web --> pedidos : HTTPS / REST
mobile --> auth : HTTPS / REST (Auth)
mobile --> pedidos : HTTPS / REST

' Interacción lógica interna (Inyección de dependencias)
auth ..> usuarios : Validación
pedidos ..> flota : Consulta capacidad
pedidos ..> auditoria : Registro de log

' Conexión única a base de datos
auth --> db_main : SQL / JPA (Esquema Auth)
usuarios --> db_main : SQL / JPA (Esquema Users)
pedidos --> db_main : SQL / JPA (Esquema Orders)
flota --> db_main : SQL / JPA (Esquema Inventory)
auditoria --> db_main : SQL / JPA (Esquema Audit)
@enduml

```